

Reminder: Project 1 is due Monday.

- If you fully digested the Stacks homework, this project should be straightforward.

We will have covered everything you need for project 2 after today.

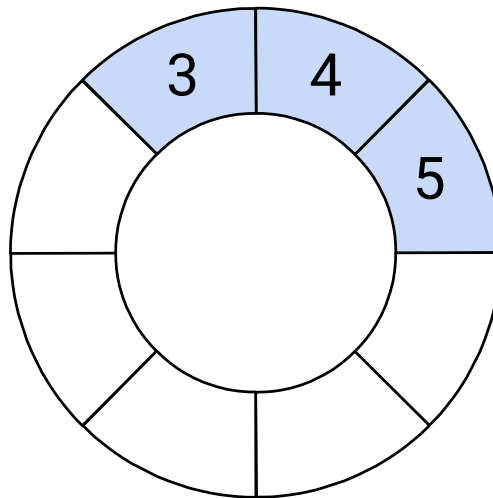
Note: The Stacks homework was new.

- Sorry if it was intimidating when you got started.
- It would have been useful to provide a bit more information on getting started.

Midterm 1: Unlike prior semesters, it will be based more heavily on homeworks and mini-projects.

- So make sure to really understand the homeworks and mini-projects.

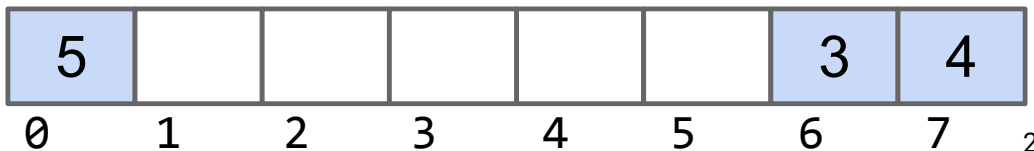
[3, 4, 5]



Lecture 7

ArrayLists

CS61B, Fall 2025 @ UC Berkeley



Where We Left off: The Naive AList

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100]; size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int getLast() {
        return items[size - 1];
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

Today, we'll cover:

- “Expanding” our array to support more than 100 items when necessary.
- Making our AList generic.
- Optimizing AList speed.

```
public int removeLast() {
    int x = items[size - 1];
    items[size - 1] = 0;
    size -= 1;
    return x;
}
```

Resizing Array Theory

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

- **Resizing Array Theory**
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

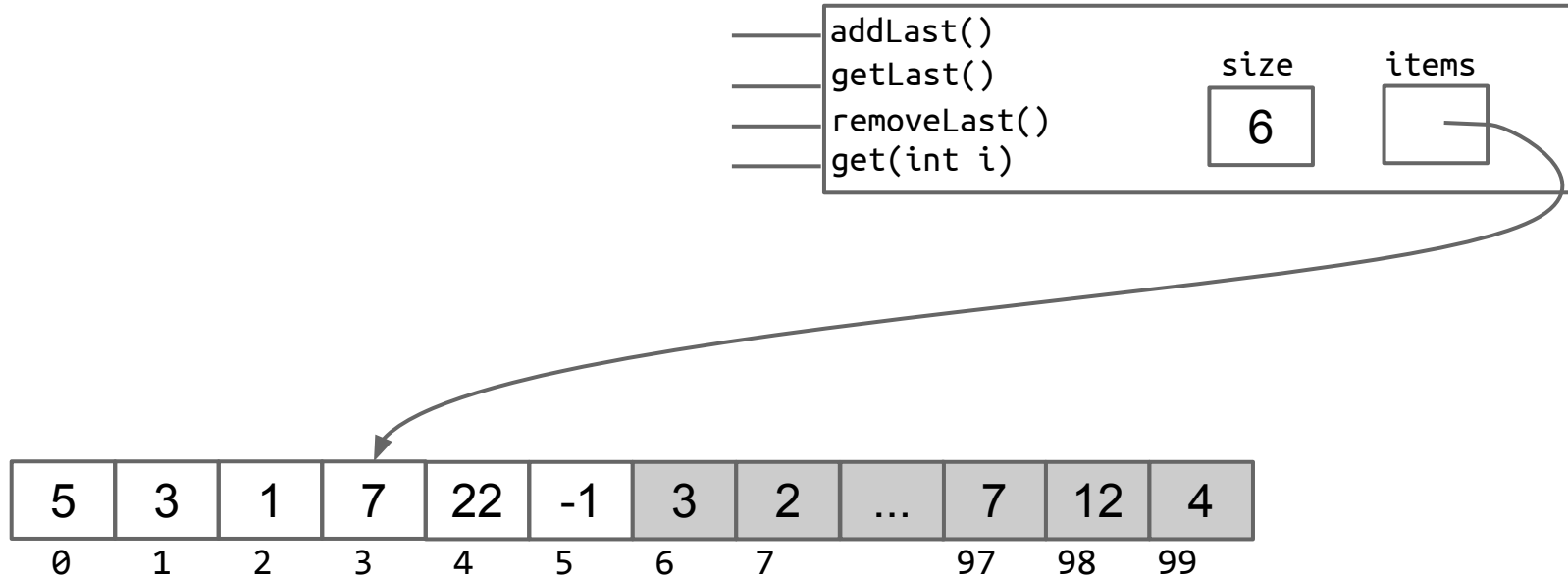
Generic ArrayLists

Circular ArrayLists

- Circular Arrays
- Linear Implementation of Circular Arrays

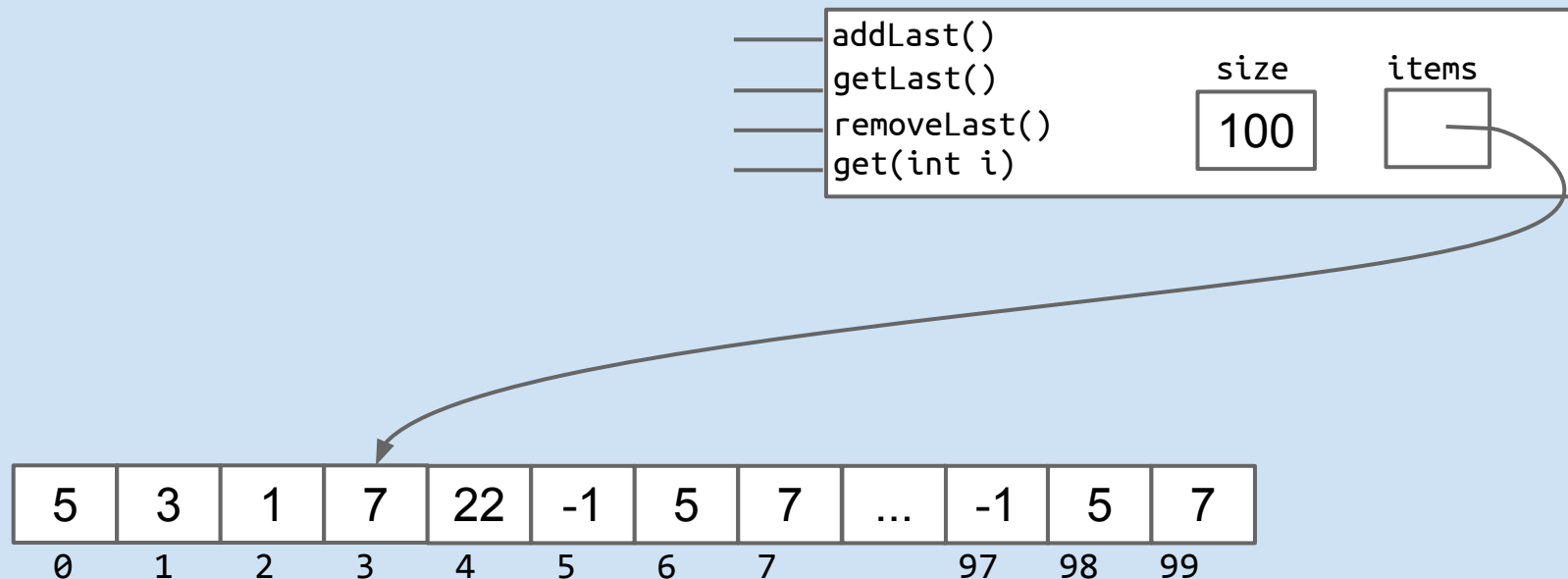
The Mighty AList

Key Idea: Use some subset of the entries of an array.



The Mighty (?) AList

Key Idea: Use some subset of the entries of an array.

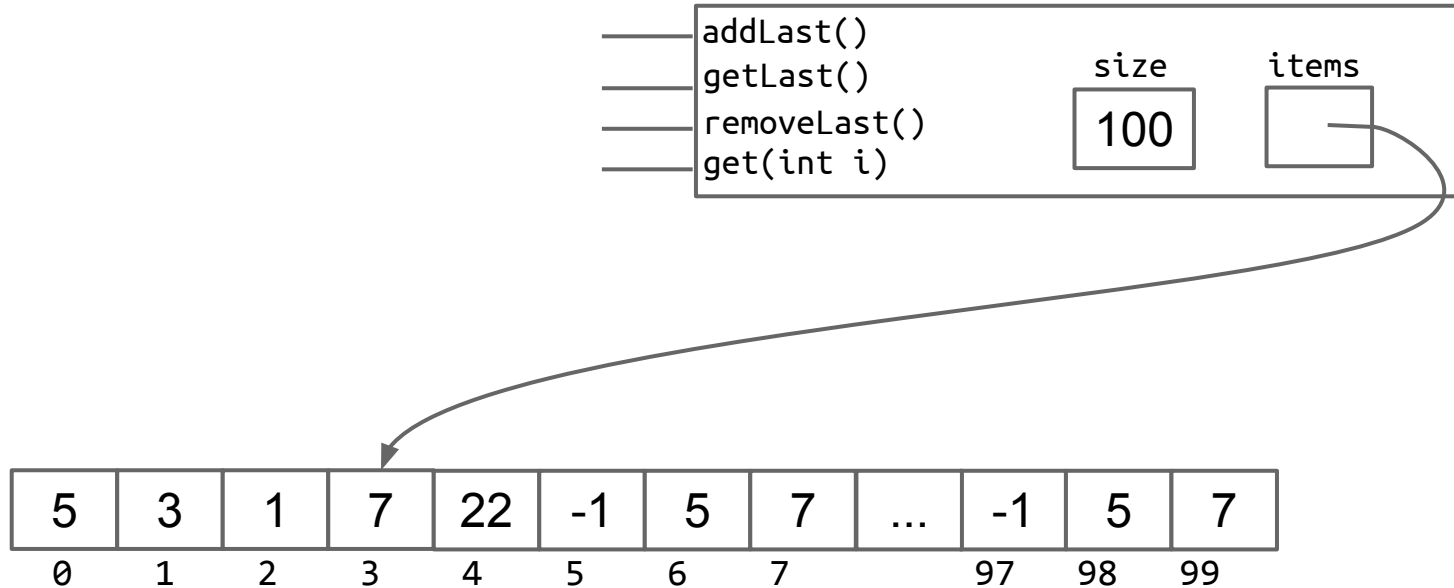


What happens if we insert into the AList above? What should we do about it?

Array Resizing

size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

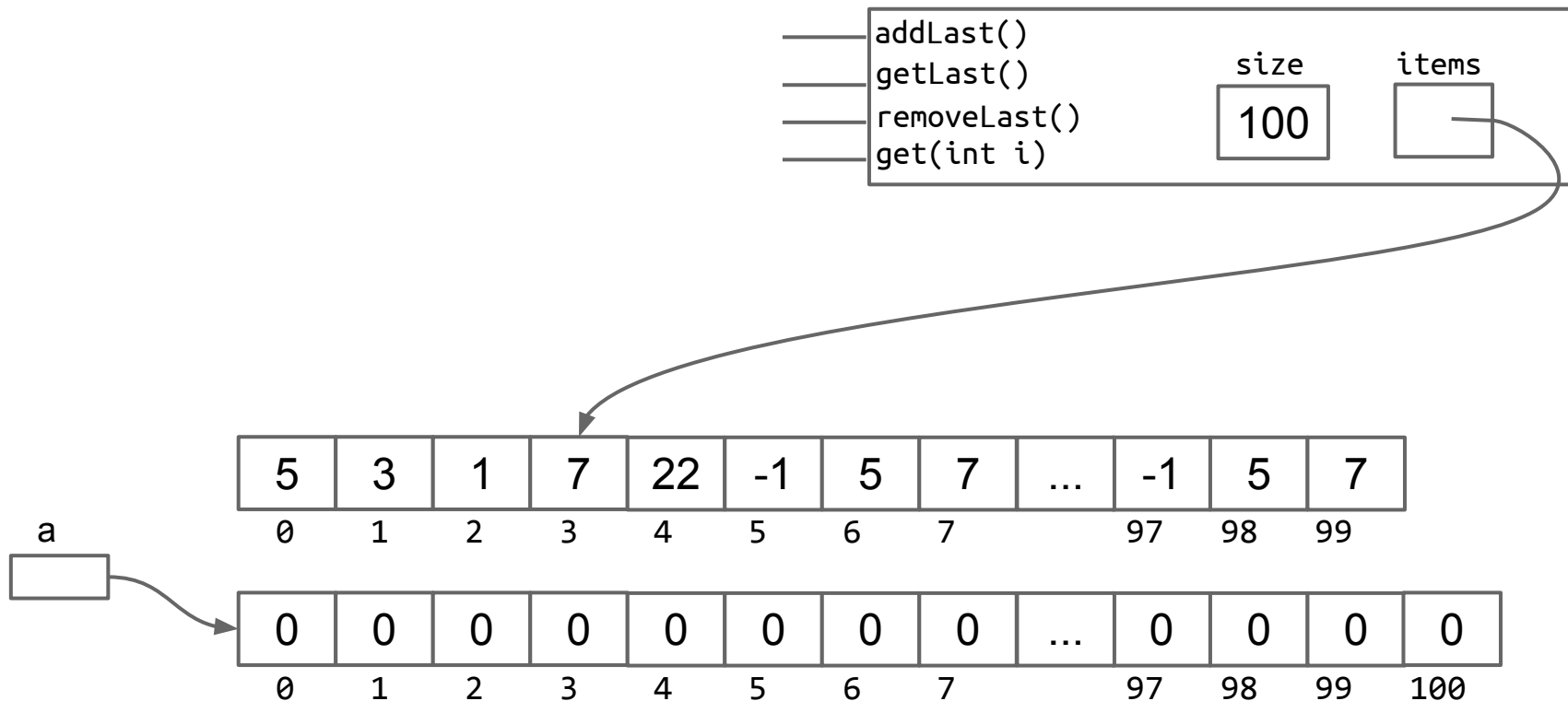


Array Resizing

size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] a = new int[size+1];`

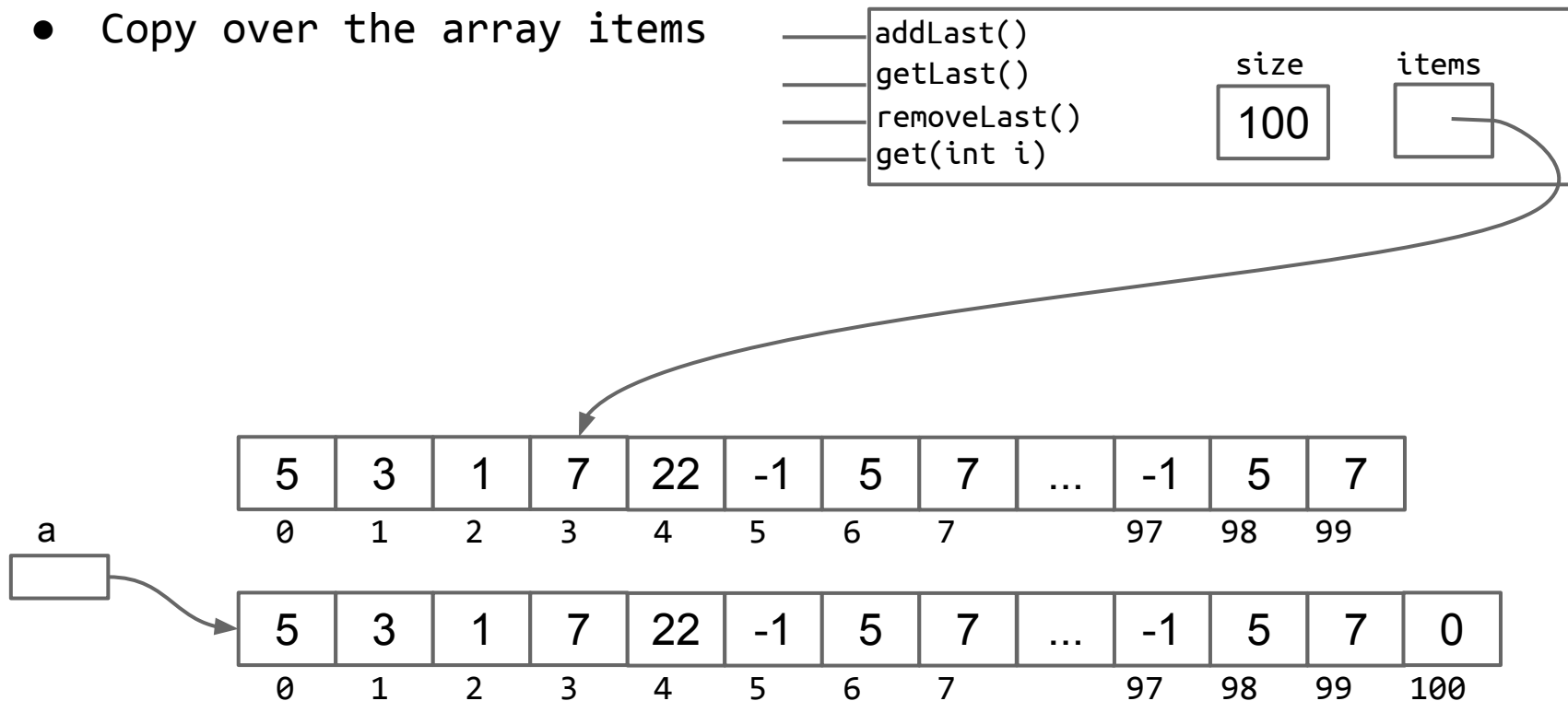


Array Resizing

size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] a = new int[size+1];`
- Copy over the array items

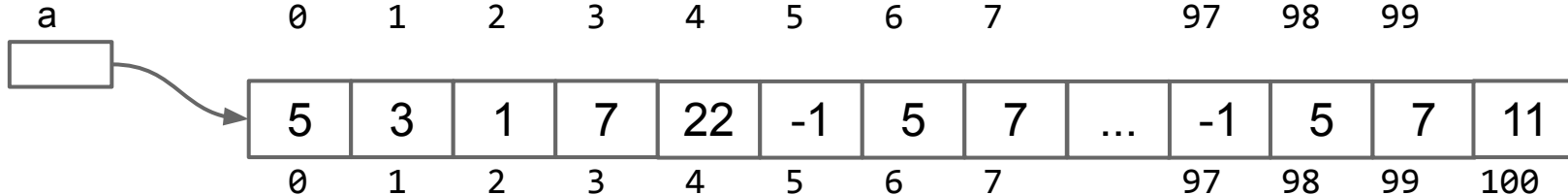
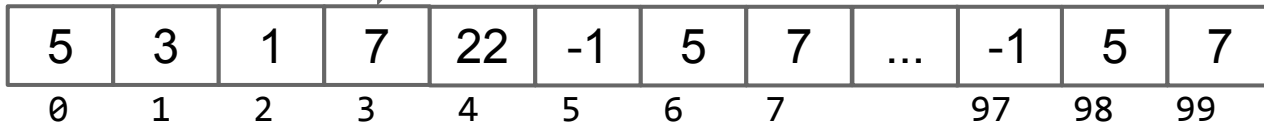


Array Resizing

size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] a = new int[size+1];`
- Copy over the array items
- `a[size] = 11;`

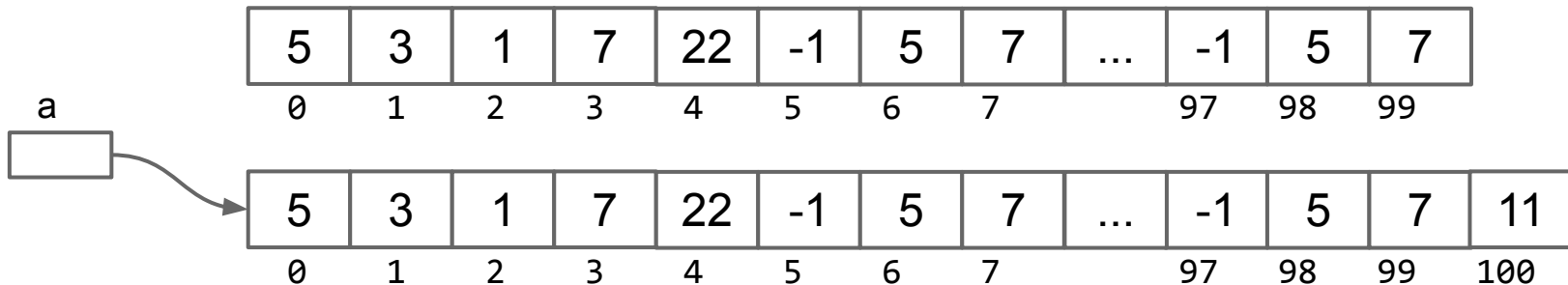


Array Resizing

size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] a = new int[size+1];`
- Copy over the array items
- `a[size] = 11;`
- `items = a;    size +=1;`



Array Resizing

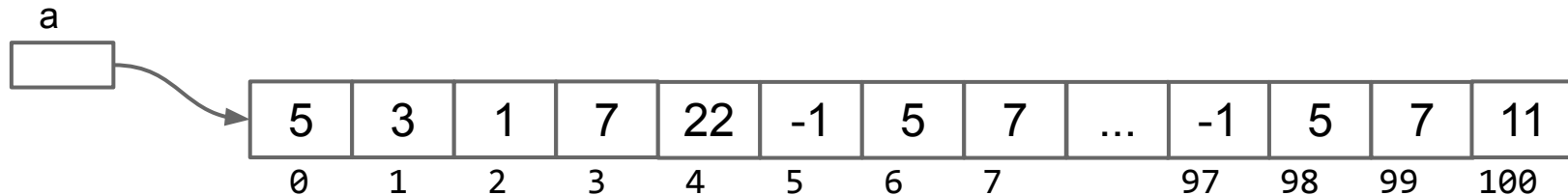
size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] a = new int[size+1];`
- Copy over the array items
- `a[size] = 11;`
- `items = a;    size +=1;`



We call this process "resizing"



Two ways to copy arrays:

- Item by item using a loop.
- Using arraycopy. Takes 5 parameters:
 - Source array
 - Start position in source
 - Target array
 - Start position in target
 - Number to copy

```
System.arraycopy(items, 0, a, 3, 2);
```

(In Python): `a[3:5] = items[0:2]`

arraycopy is (likely to be) faster, particularly for large arrays. More compact code.

- Code is (arguably) harder to read.
- Recommendation: Don't use on project 2 (it's a trap for reasons we'll see later today!).

Resizing Array Implementation

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

- Resizing Array Theory
- **Resizing Array Implementation**
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Circular ArrayLists

- Circular Arrays
- Linear Implementation of Circular Arrays

Let's implement the resizing capability.

- As usual, for those of you watching online, I recommend trying to implement this on your own before watching me do it.
- Starter code is provided in the `lec7_lists4/resizeExercise` folder in the `lectureCode-fa25` repo if you want to try it out on a computer.
 - You'll want to write code so that the `add200Items` test in `AListTest` works.

AList.java

```
public class AList {  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
  
        items[size] = x;  
        size += 1;  
    }  
}
```


Coding Demo: Resizing Array

AList.java

```
public class AList {  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
  
            }  
            items[size] = x;  
            size += 1;  
        }  
    }  
}
```

Coding Demo: Resizing Array

AList.java

```
public class AList {  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
            int[] a = new int[size + 1];  
  
            }  
        items[size] = x;  
        size += 1;  
    }  
}
```

Coding Demo: Resizing Array

AList.java

```
public class AList {  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
            int[] a = new int[size + 1];  
            System.arraycopy(items, 0, a, 0, size);  
        }  
        items[size] = x;  
        size += 1;  
    }  
}
```

Coding Demo: Resizing Array

AList.java

```
public class AList {  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
            int[] a = new int[size + 1];  
            System.arraycopy(items, 0, a, 0, size);  
            items = a;  
        }  
        items[size] = x;  
        size += 1;  
    }  
}
```

Coding Demo: Resizing Array

AList.java

```
public class AList {  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
            int[] a = new int[size + 1];  
            System.arraycopy(items, 0, a, 0, size);  
            items = a;  
        }  
        items[size] = x;  
        size += 1;  
    }  
}
```

The resizing functionality is really its own independent operation, separate from addLast.

We could organize our code better by moving this code into its own function.

Coding Demo: Resizing Array

AList.java

```
public class AList {  
    /** Resizes the underlying array to the target capacity. */  
    private void resize(int capacity) {  
  
    }  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
            int[] a = new int[size + 1];  
            System.arraycopy(items, 0, a, 0, size);  
            items = a;  
        }  
        items[size] = x;  
        size += 1;  
    }  
}
```

Coding Demo: Resizing Array

AList.java

```
public class AList {  
    /** Resizes the underlying array to the target capacity. */  
    private void resize(int capacity) {  
        int[] a = new int[size + 1];  
        System.arraycopy(items, 0, a, 0, size);  
        items = a;  
    }  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
  
            }  
        items[size] = x;  
        size += 1;  
    }  
}
```

Coding Demo: Resizing Array

AList.java

```
public class AList {  
    /** Resizes the underlying array to the target capacity. */  
    private void resize(int capacity) {  
        int[] a = new int[capacity];  
        System.arraycopy(items, 0, a, 0, size);  
        items = a;  
    }  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
  
            }  
        items[size] = x;  
        size += 1;  
    }  
}
```


Coding Demo: Resizing Array

AList.java

```
public class AList {  
    /** Resizes the underlying array to the target capacity. */  
    private void resize(int capacity) {  
        int[] a = new int[capacity];  
        System.arraycopy(items, 0, a, 0, size);  
        items = a;  
    }  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
            resize(size + 1);  
        }  
        items[size] = x;  
        size += 1;  
    }  
}
```

Resizing Array Code

```
public void addLast(int x) {  
    if (size == items.length) {  
        int[] a = new int[size + 1];  
        System.arraycopy(items, 0, a, 0, size);  
        items = a;  
    }  
    items[size] = x;  
    size += 1;  
}
```

Works

```
private void resize(int capacity) {  
    int[] a = new int[capacity];  
    System.arraycopy(items, 0, a, 0, size);  
    items = a;  
}  
  
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + 1);  
    }  
    items[size] = x;  
    size += 1;  
}
```

Much Better

Easier to read and understand.
Easier to test the correctness of each function.

Willful Obscurantism

Lecture 7, CS61B, Fall 2025

A Last Look at Linked Lists

Naive ArrayLists

- Basic ArrayList Implementation
- The Allegory of the Cave
- removeLast Implementation

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- **Runtime Analysis (Warmup)**
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Obscurantism in Java

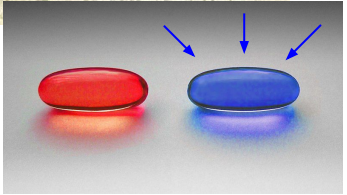
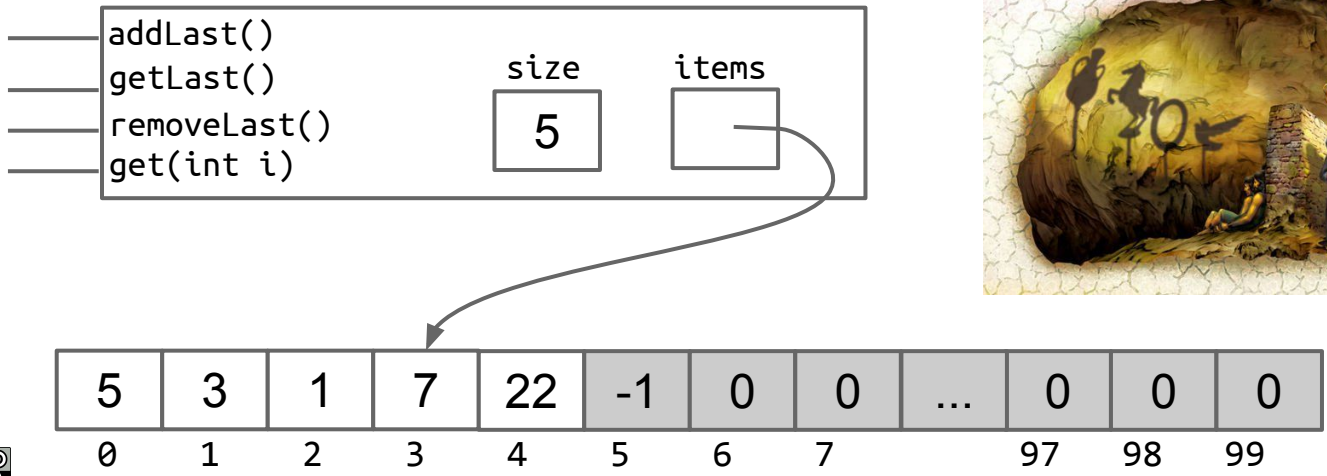
One last thought: Obscurantism in Java

We talk of “layers of abstraction” often in computer science.

- Related concept, **obscurantism**: “the practice of deliberately preventing the facts or full details of something from becoming known.”
- The user of a class does not and should not know how it works.

User’s mental model: {5, 3, 1, 7, 22, -1} → {5, 3, 1, 7, 22}

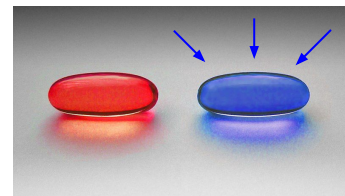
Actual truth:



One last thought: Obscurantism in Java

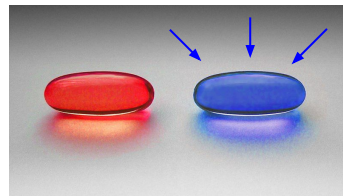
We talk of “layers of abstraction” often in computer science.

- Related concept: **obscurantism**. The user of a class does not and should not know how it works.
 - The Java language allows you to enforce this with ideas like **private**!



We talk of “layers of abstraction” often in computer science.

- Related concept: **obscurantism**. The user of a class does not and should not know how it works.
 - The Java language allows you to enforce this with ideas like **private**!
- A good programmer acts if unnecessary details are obscure, even within a class.
 - Example: `addFirst` and `resize` should be written totally independently. You should not be thinking about the details of one method while writing the other. Simply trust that the other works.
 - Breaking programming tasks down into small pieces (especially functions) helps with this greatly!
 - Through judicious use of testing, we can build confidence in these small pieces, as we saw in lecture 6.



Runtime Analysis (Warmup)

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- **Runtime Analysis (Warmup)**
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Circular ArrayLists

- Circular Arrays
- Linear Implementation of Circular Arrays

Suppose we have a full array of size 100. If we call `addLast` two times, how many **total** array memory boxes will we need to create and fill (for just these 2 calls)?

- A. 0
- B. 101
- C. 203
- D. 10,302

Bonus question: What is the maximum number of array boxes that Java will track at any given time? Assume that “garbage collection” happens immediately when all references to an object are lost.

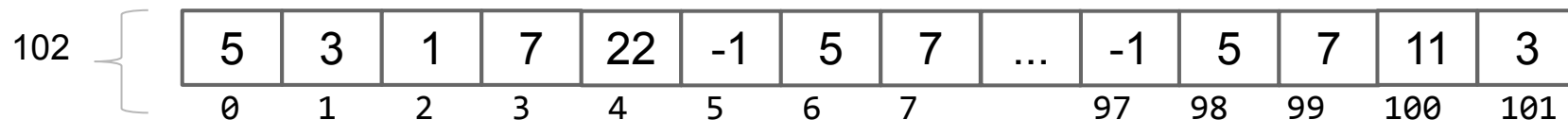
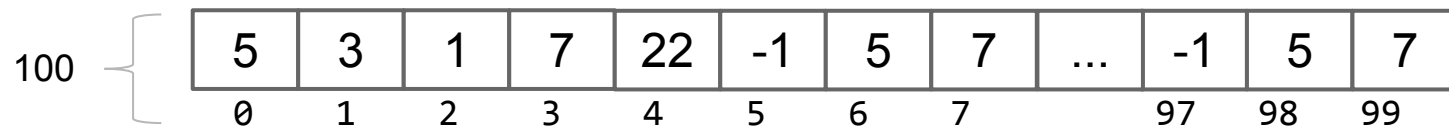
```
private void resize(int capacity) {  
    int[] a = new int[capacity];  
    System.arraycopy(items, 0, a, 0, size);  
    items = a;  
}  
  
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + 1);  
    }  
    items[size] = x;  
    size += 1;  
}
```



Array Resizing

Resizing twice requires us to create and fill 203 total memory boxes.

- Bonus answer: Most boxes at any one time is 203.
- When the second addLast is done, we are left with 102 boxes.



Runtime Analysis

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- **Runtime Analysis**
- Better Resizing Strategy

Generic ArrayLists

Circular ArrayLists

- Circular Arrays
- Linear Implementation of Circular Arrays

Demo: Speed Testing the ArrayList

```
jug ~/.../speedtest  
$ java SpeedTestSLList
```

```
Total runtime: 3 ms
```

SpeedTestSLList.java

```
public class SpeedTestSLList {  
    public static void main(String[] args) {  
        long startTime = System.currentTimeMillis();  
  
        SLList<Integer> L = new SLList<>();  
        int i = 0;  
        while (i < 100000) {  
            L.addFirst(i);  
            i = i + 1;  
        }  
  
        long endTime = System.currentTimeMillis();  
        System.out.println("Total runtime: "  
                           + (endTime - startTime) + " ms");  
    }  
}
```

Adding 100,000 items to a new SLList is very fast (order of milliseconds).

Demo: Speed Testing the ArrayList

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

```
Total runtime: 602 ms
```

SpeedTestSLList.java

```
public class SpeedTestAList {  
    public static void main(String[] args) {  
        long startTime = System.currentTimeMillis();  
  
        AList L = new AList();  
        int i = 0;  
        while (i < 100000) {  
            L.addLast(i);  
            i = i + 1;  
        }  
  
        long endTime = System.currentTimeMillis();  
        System.out.println("Total runtime: "  
            + (endTime - startTime) + " ms");  
    }  
}
```

Adding 100,000 items to a new AList is slow (order of seconds).

Suppose we have a full array of size 100. If we call `addLast` until `size = 1000`, roughly how many total array memory boxes will we need to create and fill?

- A. 1,000
- B. 500,000
- C. 1,000,000
- D. 500,000,000,000
- E. 1,000,000,000,000

Bonus question: What is the maximum number of array boxes that Java will track at any given time? Assume that “garbage collection” happens immediately when all references to an object are lost.

```
private void resize(int capacity) {  
    int[] a = new int[capacity];  
    System.arraycopy(items, 0, a, 0, size);  
    items = a;  
}  
  
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + 1);  
    }  
    items[size] = x;  
    size += 1;  
}
```



Suppose we have a full array of size 100. If we call `addLast` until `size = 1000`, roughly how many total array memory boxes will we need to create and fill?

B. 500,000

Going from capacity 100 to 101: 101

From 101 to 102: 102

...

From: 999 to 1000: 1000

```
private void resize(int capacity) {  
    int[] a = new int[capacity];  
    System.arraycopy(items, 0, a, 0, size);  
    items = a;  
}
```

We'll be doing a lot of this after the midterm.

Total array boxes created/copied: $101 + 102 + \dots + 1000$

Since sum of $1 + 2 + 3 + \dots + N = N(N+1)/2$, $\text{sum}(101, \dots, 1000)$ is close to 500,000.

See: <http://mathandmultimedia.com/2010/09/15/sum-first-n-positive-integers>

SLList vs. AList Timing Tables

Below we see the time to insert N items for SLLists and ALists.

```
jug ~/.../speedtest  
$ java SpeedTestSLListTable
```

N	Time (ms)
10000	1
100000	3
1000000	71
10000000	875

```
jug ~/.../speedtest  
$ java SpeedTestAListTable
```

N	Time (ms)
10000	19
100000	903
1000000	70227

Runtime and Space Usage Analysis

Since sum of $1 + 2 + 3 + \dots + N = N(N+1)/2$, $\text{sum}(101, \dots, 1000)$ is close to 500,000.

Rough intuition: Form pairs that all sum to $N+1$:

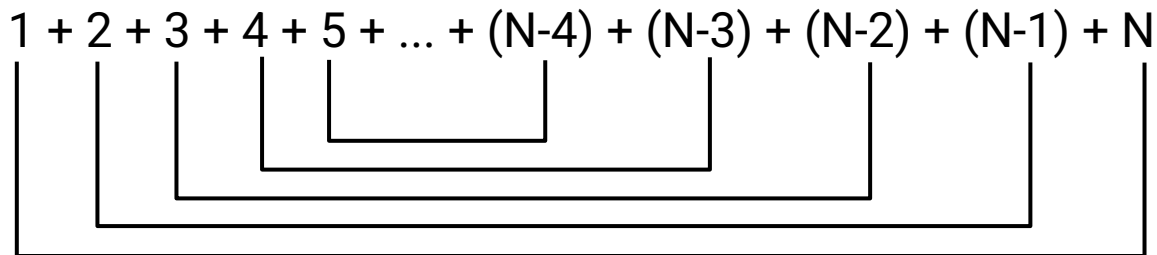
$N + 1$

$(N-1) + 2$

$(N-2) + 3$

...

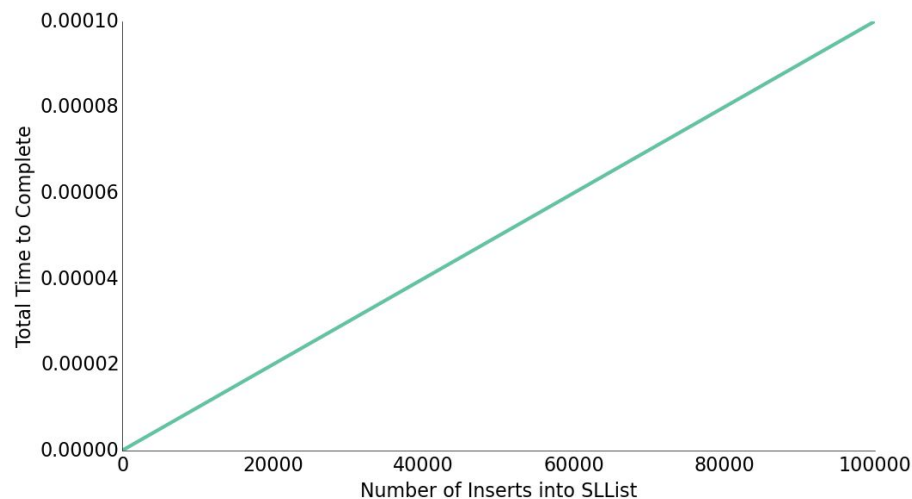
There are $N/2$ such pairs.



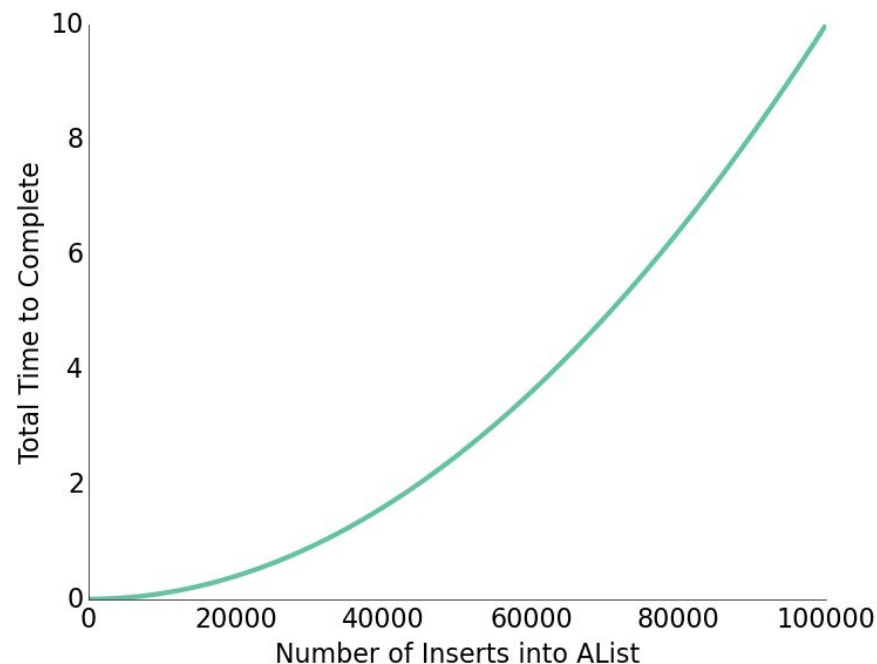
Resizing Slowness (Theoretical Plots)

Inserting 100,000 items requires roughly 5,000,000,000 new containers.

- Computers operate at the speed of GHz (due billions of things per second).
- No huge surprise that 100,000 items took roughly seconds.



Note: Insert here is addFirst



Resizing Slowness (Theoretical Plots)

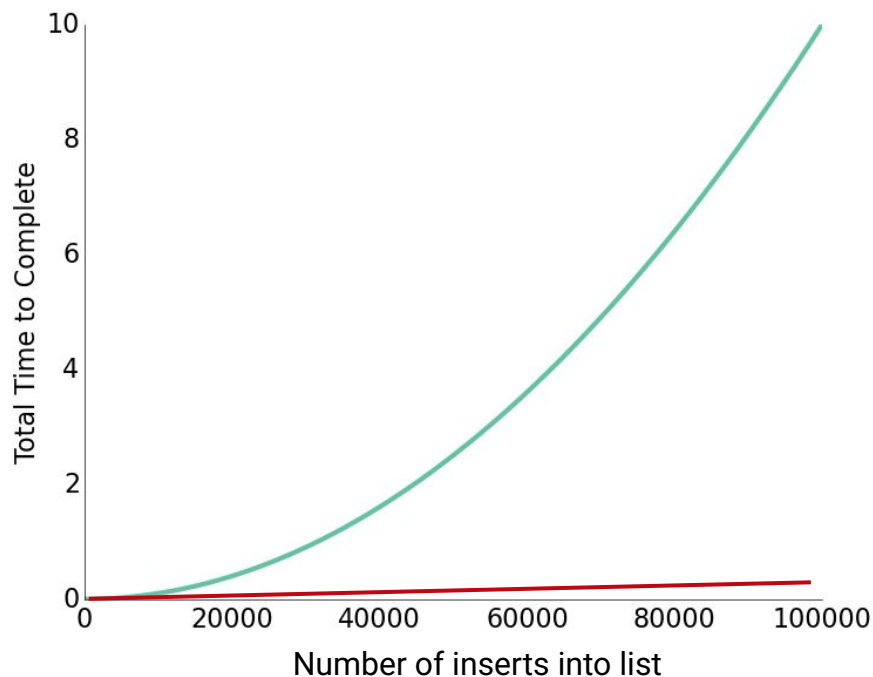
Inserting 100,000 items requires roughly 5,000,000,000 new containers.

- Computers operate at the speed of GHz (due billions of things per second).
- No huge surprise that 100,000 items took roughly seconds.

The same graphs from the previous slide, placed on top of each other.

Red line = SLList

Teal line = AList



Better Resizing Strategy

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- **Better Resizing Strategy**

Generic ArrayLists

Circular ArrayLists

- Circular Arrays
- Linear Implementation of Circular Arrays

Fixing the Resizing Performance Bug

How do we fix this?

```
private void resize(int capacity) {  
    int[] a = new int[capacity];  
    System.arraycopy(items, 0, a, 0, size);  
    items = a;  
}  
  
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + 1);  
    }  
    items[size] = x;  
    size += 1;  
}
```

Demo: Speed Testing the ArrayList

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + 1000);  
    }  
    items[size] = x;  
    size += 1;  
}
```

```
jug ~/.../speedtest  
$ java SpeedTestArrayList
```

Total runtime: 6 ms

N=100,000

Resizing by 1000 elements instead
of 1 seems to speed up adding
100,000 items...

```
jug ~/.../speedtest  
$ java SpeedTestArrayList
```

Total runtime: 6097 ms

N=10,000,000

...but adding 10,000,000 items is
still super slow.

Demo: Speed Testing the ArrayList

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size * 2);  
    }  
    items[size] = x;  
    size += 1;  
}
```

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

Total runtime: 2 ms

N=100,000

If we double the array capacity
every time it's full, then adding
100,000 items is fast...

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

Total runtime: 25 ms

N=10,000,000

...and adding 10,000,000 items is
also fast.

(Probably) Surprising Fact

Geometric resizing is much faster.

We can't prove this until later. [See this video for a more detailed analysis.](#)

Rough intuition: As the array grows larger, we resize less often.

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + RFACTOR);  
    }  
    items[size] = x;  
    size += 1;  
}
```

Great performance. 

This is how the Python list is implemented.

 Unusably bad.

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size * RFACTOR);  
    }  
    items[size] = x;  
    size += 1;  
}
```

Suppose we have a very rare situation occur which causes us to:

- Insert 1,000,000,000 items.
- Then remove 990,000,000 items.

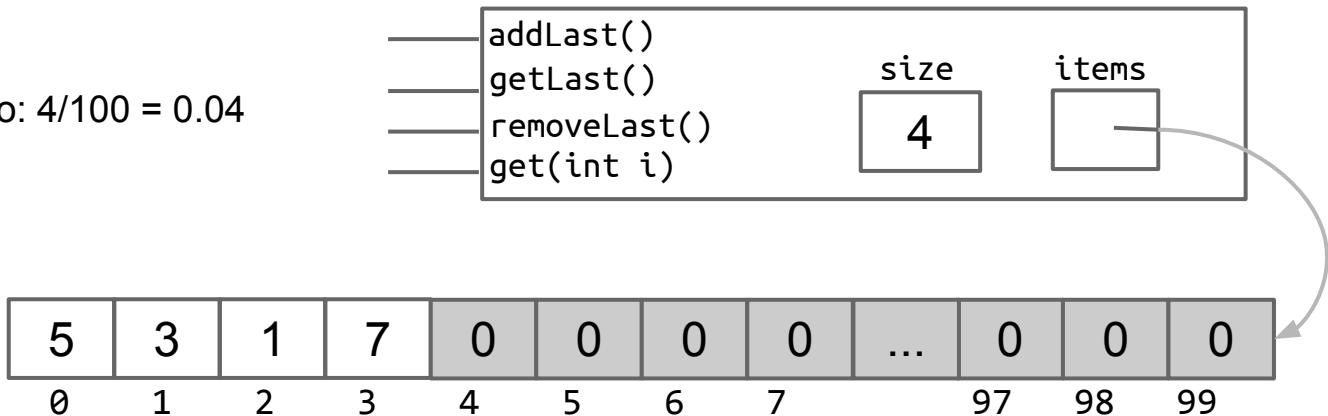
Our data structure will execute these operations acceptably fast, but afterwards there is a problem.

- What is the problem?

An AList should not only be efficient in time, but also efficient in space.

- Define the “usage ratio” $R = \text{size} / \text{items.length}$;
- Typical solution: Half array size when $R < 0.25$.
- More details in a few weeks.

Usage ratio: $4/100 = 0.04$



Later we will consider tradeoffs between time and space efficiency for a variety of algorithms and data structures.

Generic ArrayLists

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Circular ArrayLists

- Circular Arrays
- Linear Implementation of Circular Arrays

Generic ALists (similar to generic SLists)

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[8];
        size = 0;
    }

    private void resize(int capacity) {
        int[] a = new int[capacity];
        System.arraycopy(items, 0,
                        a, 0, size);
        items = a;
    }

    public int get(int i) {
        return items[i];
    }
    ...
}
```

```
public class AList<Glorp> {
    private Glorp[] items;
    private int size;

    public AList() {
        items = (Glorp[]) new Object[8];
        size = 0;
    }

    private void resize(int cap) {
        Glorp[] a = (Glorp[]) new Object[cap];
        System.arraycopy(items, 0,
                        a, 0, size);
        items = a;
    }

    public Glorp get(int i) {
        return items[i];
    }
    ...
}
```

Generic ALists (similar to generic SLists)

```
public class AList<Glorp> {
    private Glorp[] items;
    private int size;

    public AList() {
        items = (Glorp[]) new Object[8];
        size = 0;
    }

    private void resize(int cap) {
        Glorp[] a = (Glorp[]) new Object[cap];
        System.arraycopy(items, 0,
            a, 0, size);
        items = a;
    }

    public Glorp get(int i) {
        return items[i];
    }
    ...
}
```

When creating an array of references to Glorps:

- `(Glorp[]) new Object[8];`
- Causes a compiler warning, which you should ignore.

Why not just `new Glorp[cap];`

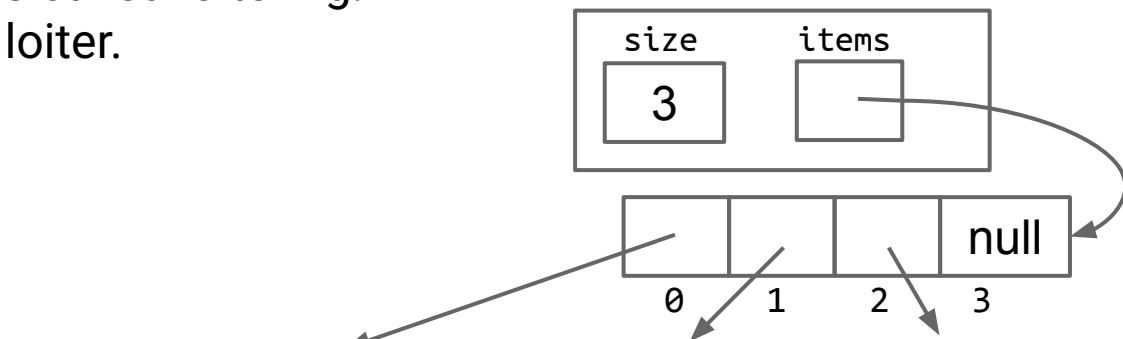
- Will cause a “generic array creation” error.

Nulling Out Deleted Items

Unlike integer based ALists, we actually want to null out deleted items.

- Java only destroys unwanted objects when the last reference has been lost.
- Keeping references to unneeded objects is sometimes called loitering.
- Save memory. Don't loiter.

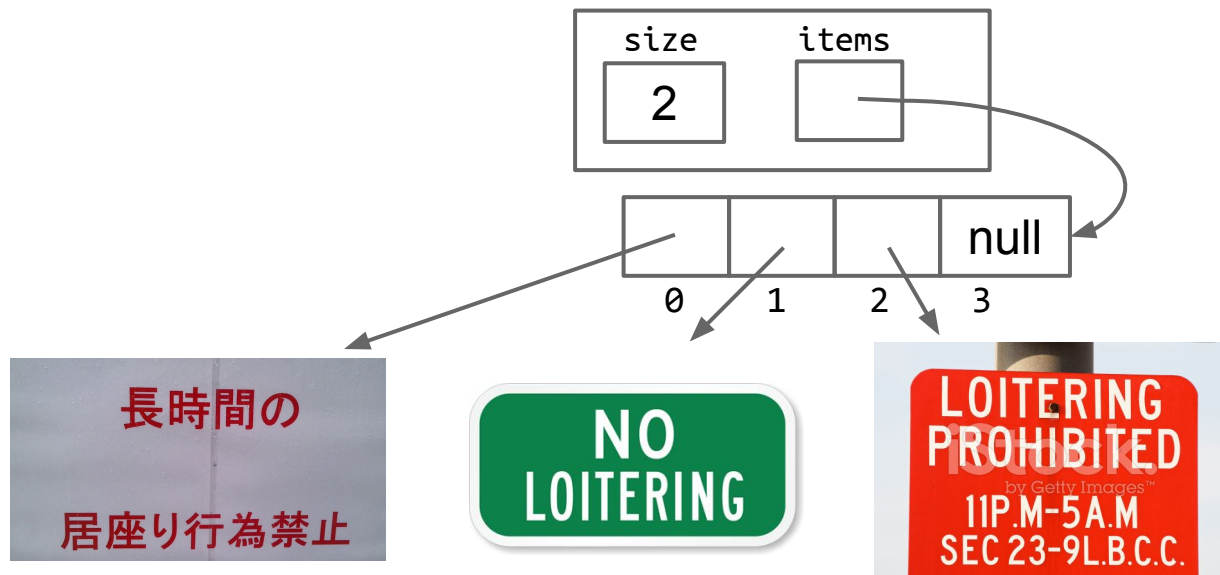
```
public Glorp removeLast() {  
    Glorp returnItem = getLast();  
    items[size - 1] = null;  
    size -= 1;  
    return returnItem;  
}
```



Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

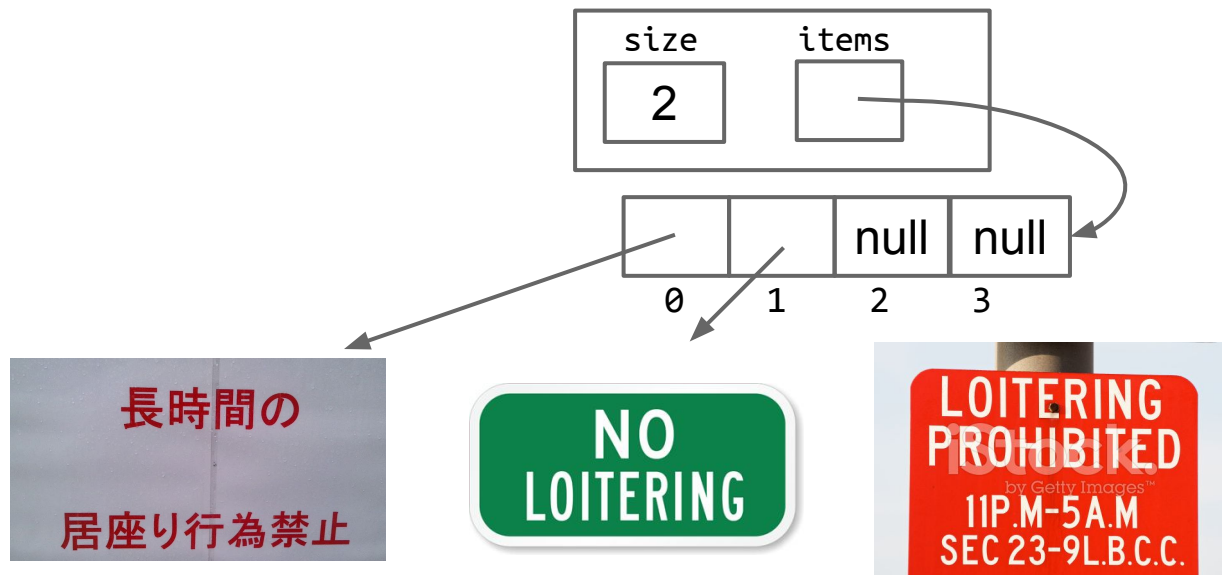


Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

By nulling out items[2], Java is free to destroy the unneeded image from memory, which could be potentially megabytes in size.

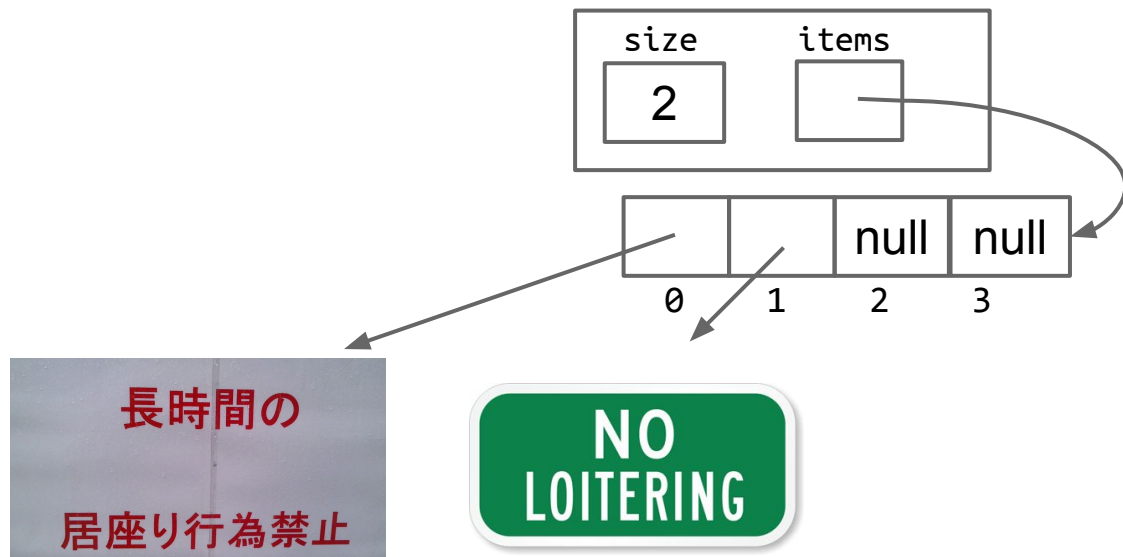


Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

By nulling out items[2], Java is free to destroy the unneeded image from memory, which could be potentially megabytes in size.



Circular Arrays

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Circular ArrayLists

- **Circular Arrays**
- Linear Implementation of Circular Arrays

What was our motivation for building an ArrayList?

What was our motivation for building an ArrayList?

- To make `get(i)` fast.

What was our motivation for building an ArrayList?

- To make `get(i)` fast.

Are ArrayLists strictly better? Is there anything a LinkedList is better at?

-

What was our motivation for building an ArrayList?

- To make `get(i)` fast.

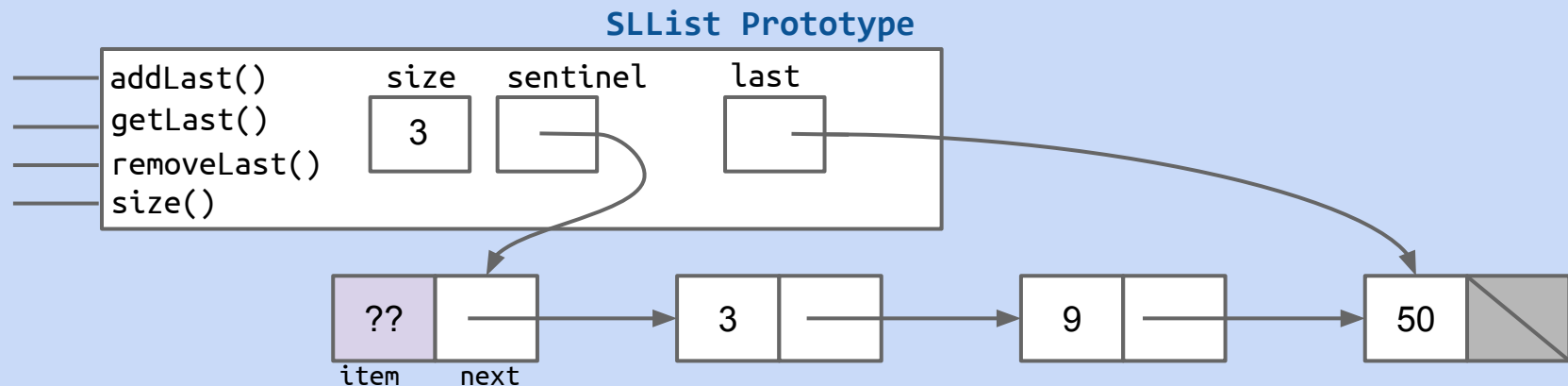
Are ArrayLists strictly better? Is there anything a LinkedList is better at?

- Yes, `addFirst` and `removeFirst` are faster for linked lists.
- Less important: LinkedList operations at ends are always fast at the ends. ArrayList operations sometimes cause a linear time resize (but negligible in practice).

Remembering Our Previous Improvement

Back when we were designing LinkedLists, we found that `removeLast()` was slow for the design below. How did we make it faster?

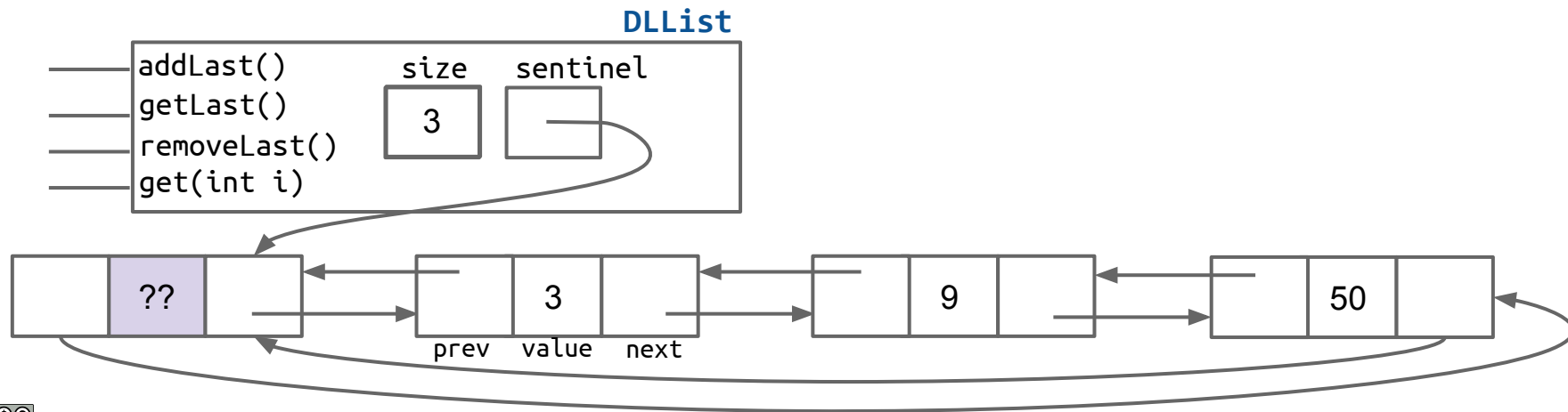
-



Remembering Our Previous Improvement

Back when we were designing LinkedLists, we found that `removeLast` was slow for the design below. How did we make it faster?

- By adding reverse links.
- This design change makes `removeLast` runtime independent of the length of the list.
 - No longer have to scan to find the second to last item.

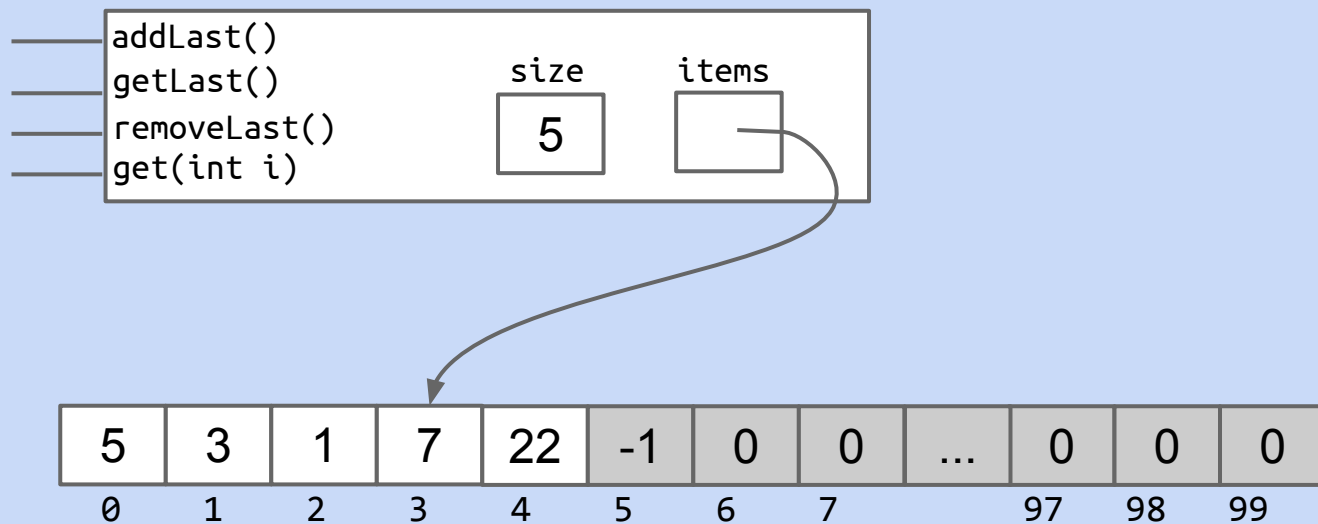


Speeding up Front Operations

For our AList class, `addFirst` or `removeFirst` are going to be slow.

- Have to shift everything to the left.

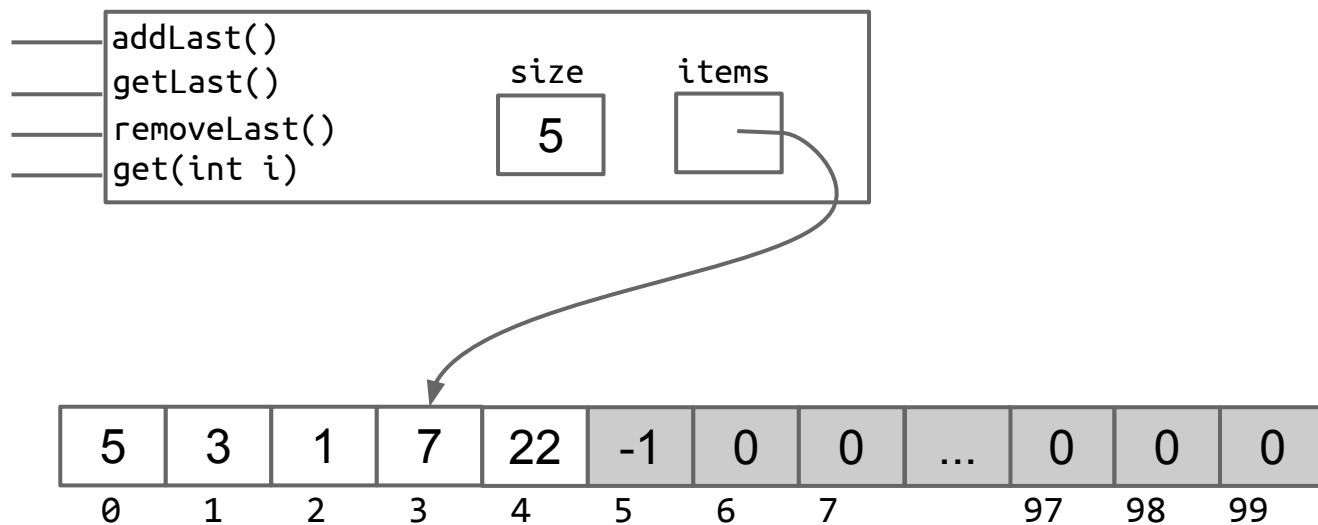
How might we modify our design so that these front operations are fast?



For our AList class, addFirst or removeFirst are going to be slow.

- Have to shift everything to the left.

How might we modify our design so that these front operations are fast?



Drop this section. The explicitly circular array is an unnecessary level of indirection.

Introducing the Circular Array

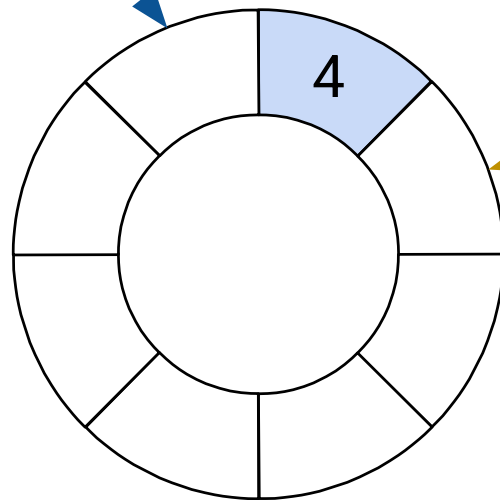
One approach: Implement the list using an array shaped like a circle.

- Suppose we start from a list containing only the number 4.

[4]

User's Mental Model

nextFirst



nextLast

Actual Circular Array

Introducing the Circular Array

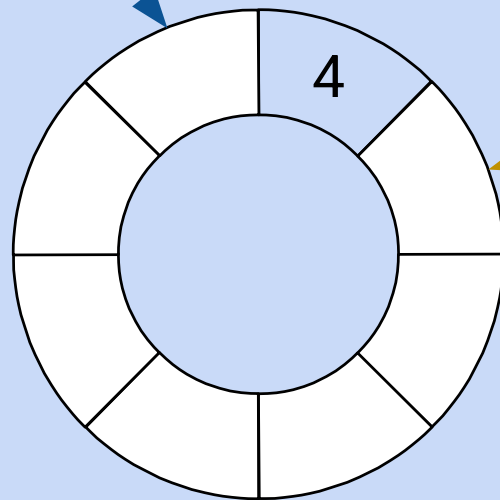
One approach: Implement the list using an array shaped like a circle.

- After calling `addFirst(3)`, how should we modify the circular array to match the user's mental model?

[3, 4]

User's Mental Model

nextFirst



Actual Circular Array

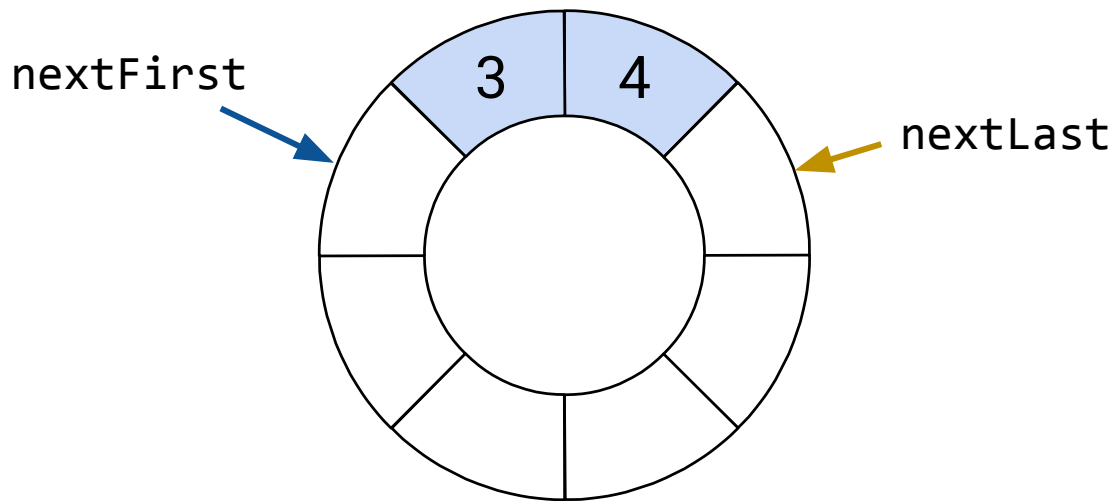
Introducing the Circular Array

One approach: Implement the list using an array shaped like a circle.

- After calling `addFirst(3)`, we put 3 in the `nextFirst` position and increment `nextFirst`.

[3, 4]

User's Mental Model



Actual Circular Array

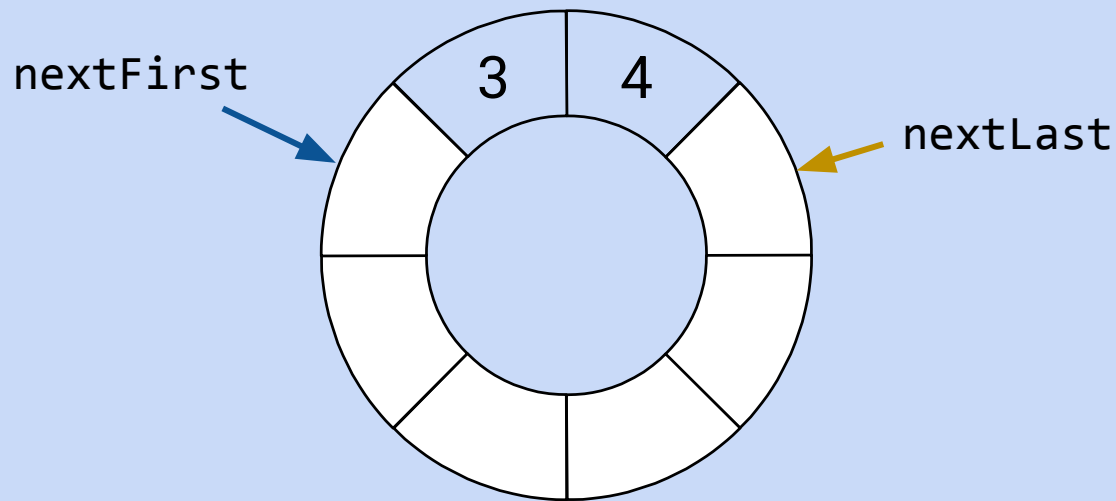
Introducing the Circular Array

One approach: Implement the list using an array shaped like a circle.

- After calling `addLast(5)`, how should we modify the circular array to match the user's mental model?

`[3, 4, 5]`

User's Mental Model



Actual Circular Array

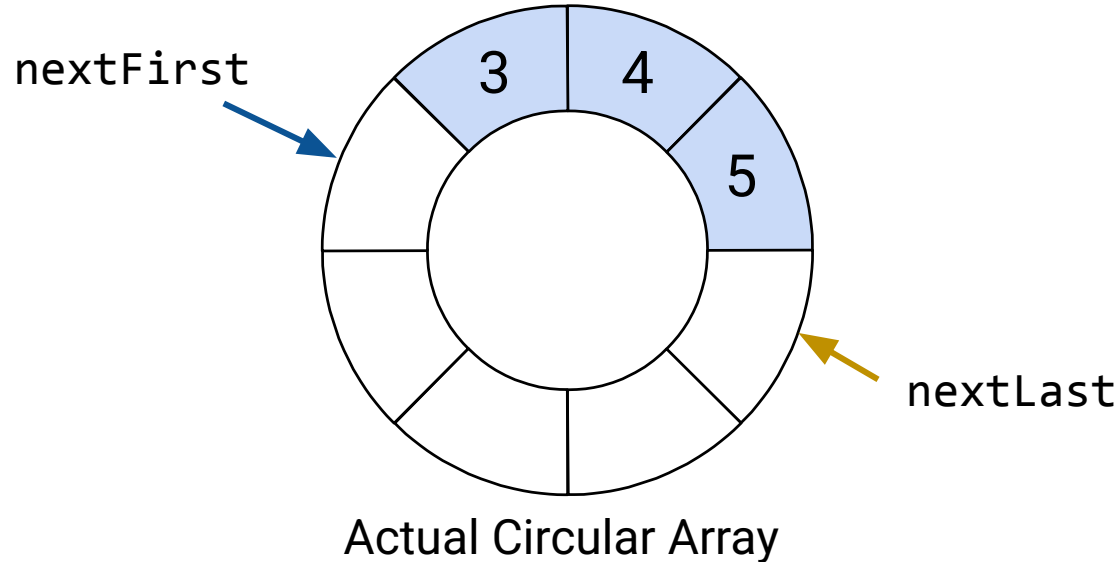
Introducing the Circular Array

One approach: Implement the list using an array shaped like a circle.

- After calling `addLast(5)`, we put 5 in the `nextLast` position and increment `nextLast`.

`[3, 4, 5]`

User's Mental Model



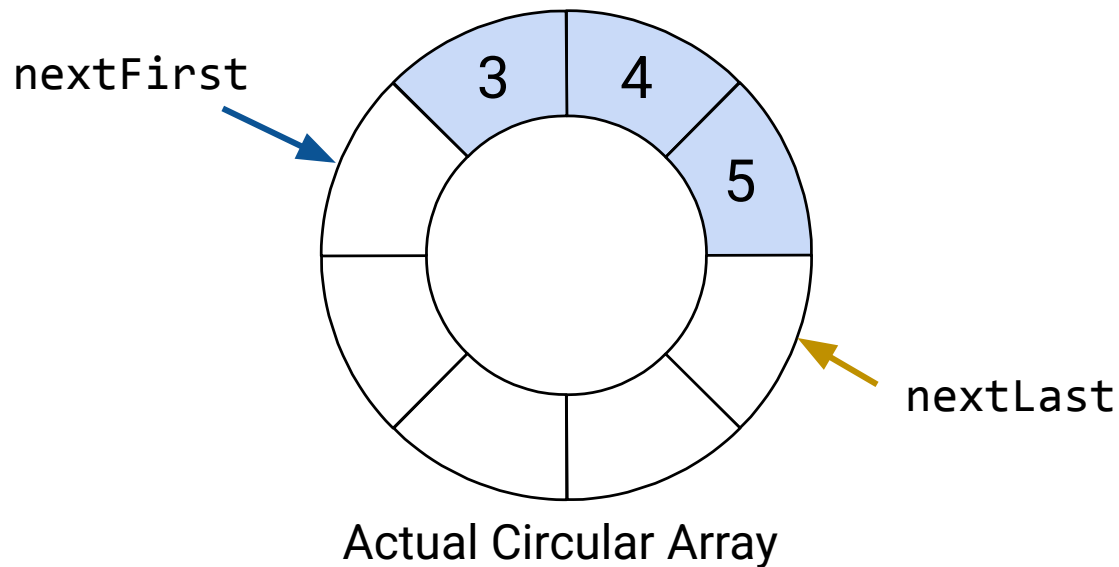
Introducing the Circular Array

One approach: Implement the list using an array shaped like a circle.

- Suppose we call `removeLast()`, try to predict what the next slide will show.

`[3, 4, 5]`

User's Mental Model



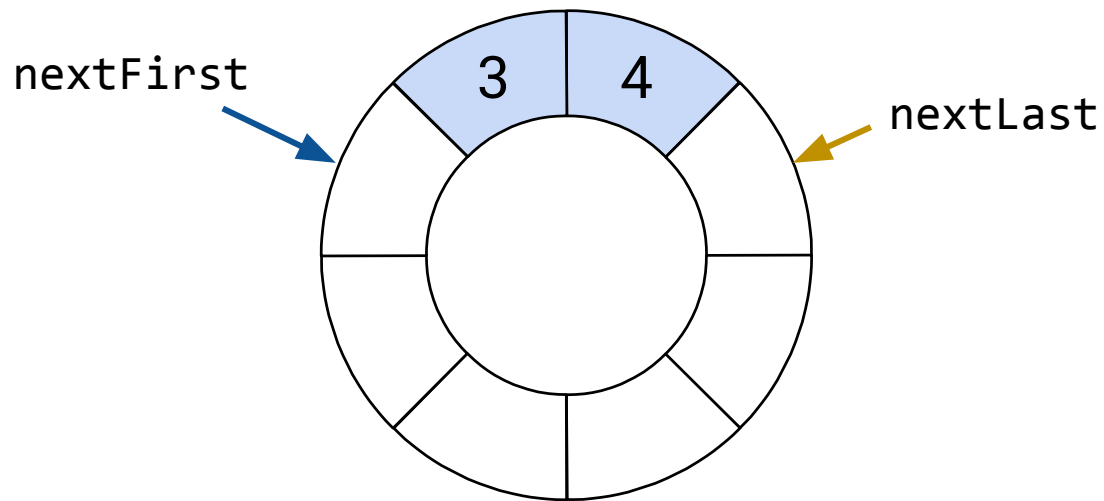
Introducing the Circular Array

One approach: Implement the list using an array shaped like a circle.

- Suppose we call `removeLast()` again, try to predict what the next slide will show.

[3, 4]

User's Mental Model



Actual Circular Array

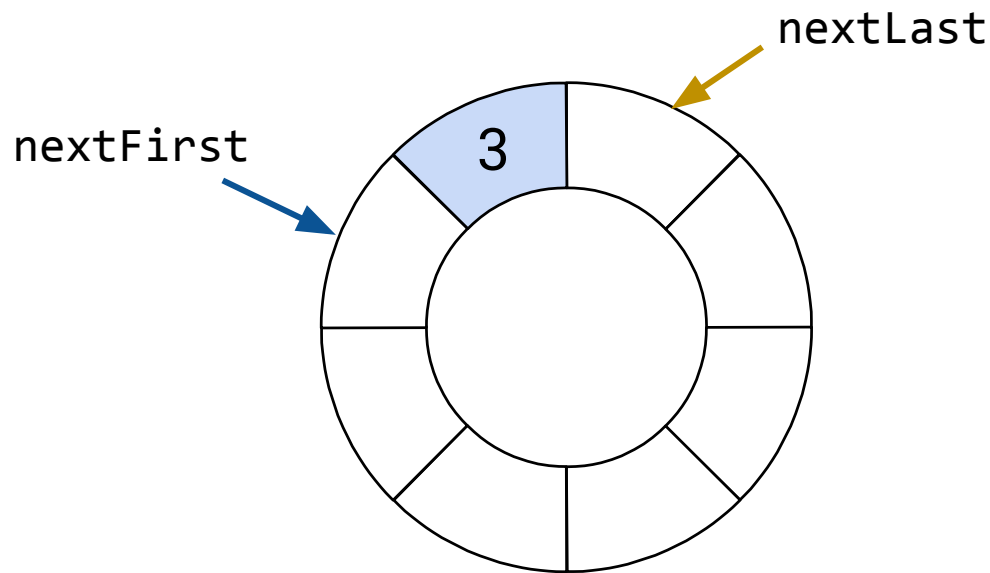
Introducing the Circular Array

One approach: Implement the list using an array shaped like a circle.

- Suppose we call `removeLast()`, try to predict what the next slide will show.

[3]

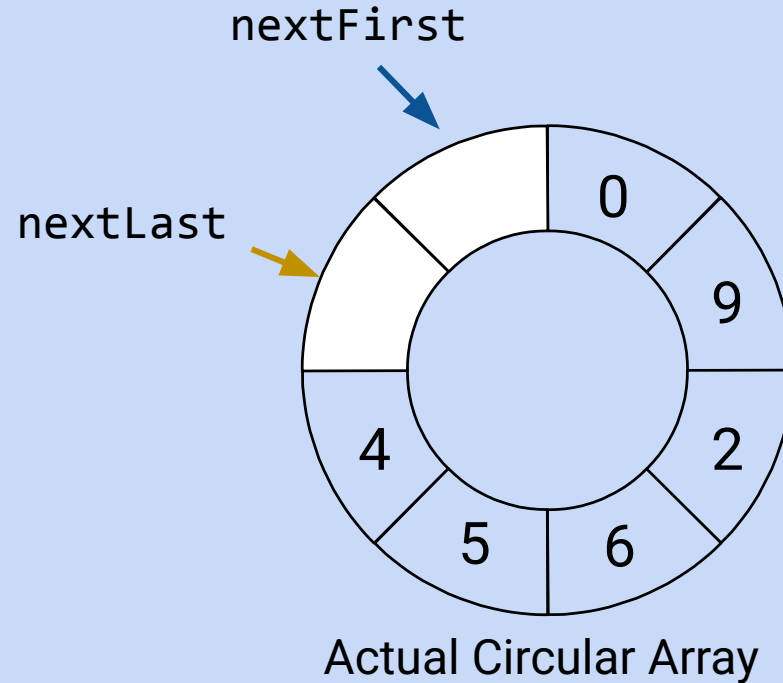
User's Mental Model



Actual Circular Array

Test Your Understanding

What mental model does the actual circular array represent below?

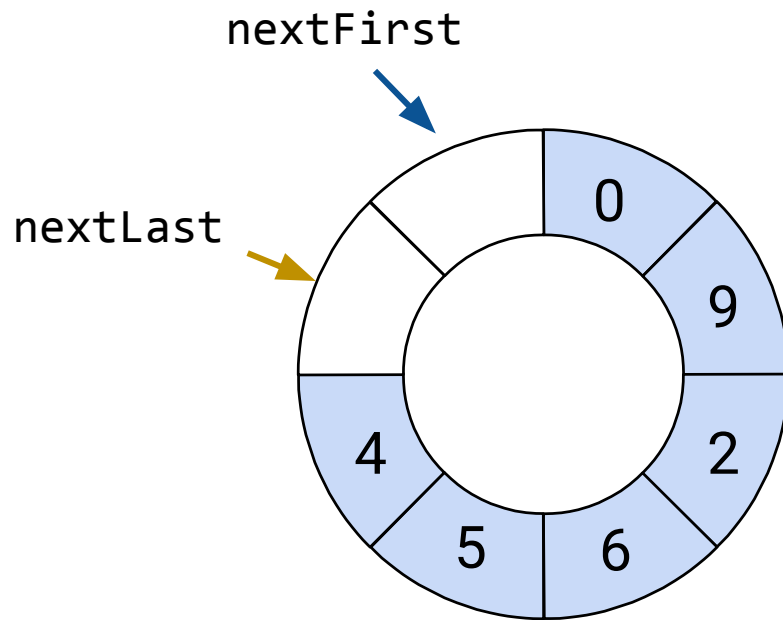


Test Your Understanding

What mental model does the actual circular array represent below?

```
[0, 9, 2, 6, 5, 4]
```

User's Mental Model



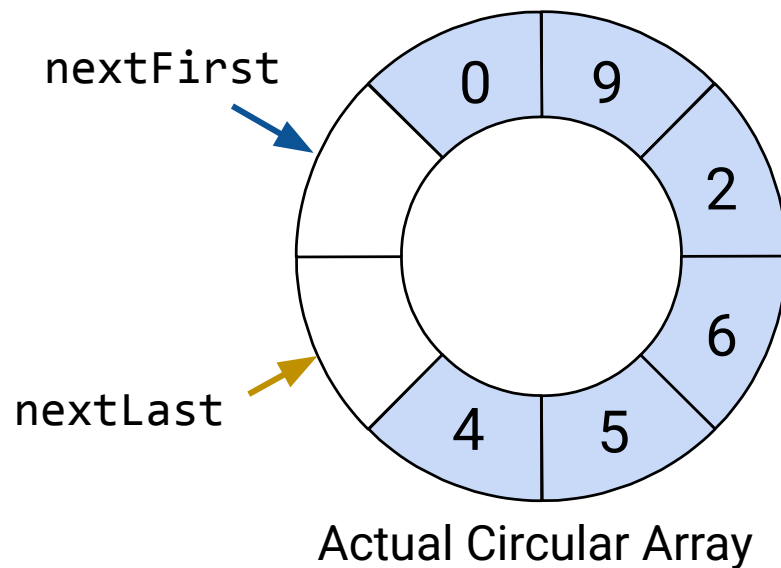
Actual Circular Array

Quick note: There are multiple circular arrays that can represent the same abstract list.

- Example: Rotating everything over counter-clockwise in the circular array yields another valid representation of the user's expectation.

[0, 9, 2, 6, 5, 4]

User's Mental Model



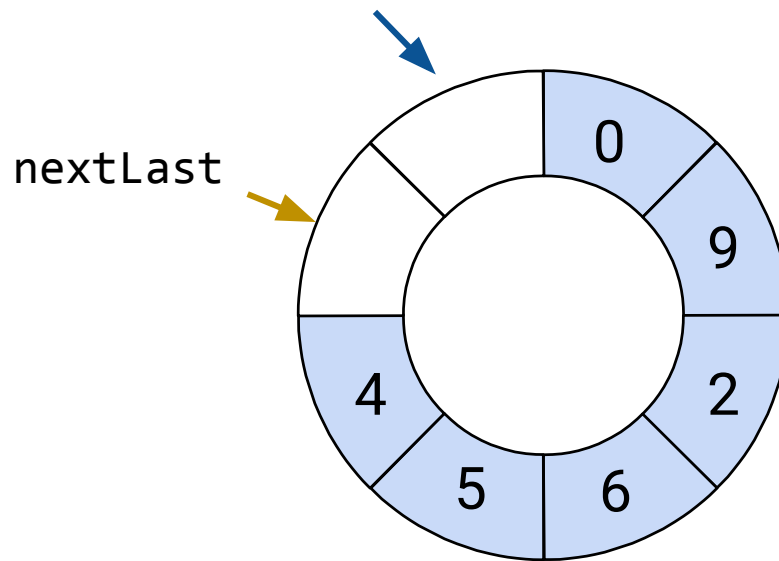
Circular Arrays

Given that we can easily convert the circular array into the user's mental model, it should be possible to write add, remove, and get operations for both ends.

- And they'll be fast!
- If we run out of room, what should we do? `nextFirst`

```
[0, 9, 2, 6, 5, 4]
```

User's Mental Model



Actual Circular Array

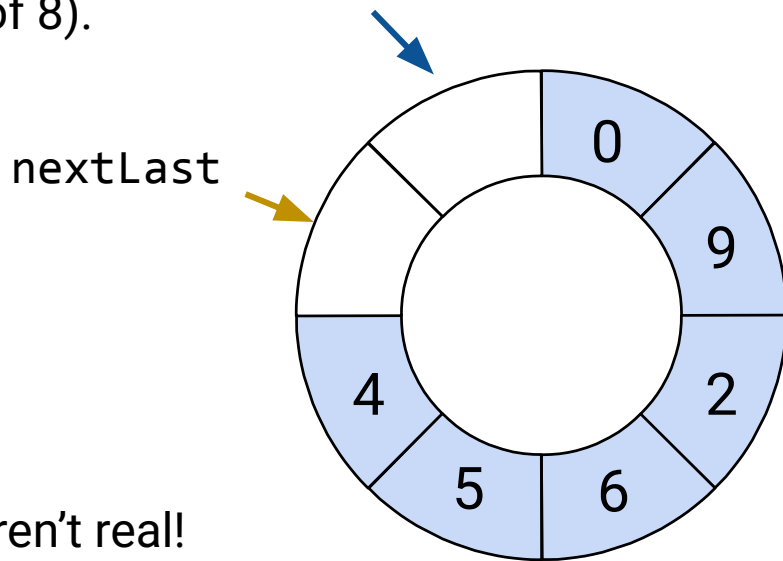
Circular Arrays

Given that we can easily convert the circular array into the user's mental model, it should be possible to write add, remove, and get operations for both ends.

- And they'll be fast!
- If we run out of room, what should we do?
 - Resize (16 sectors instead of 8).

```
[0, 9, 2, 6, 5, 4]
```

User's Mental Model



Actual Circular Array

One minor problem: Circular arrays aren't real!

Linear Implementation of Circular Arrays

Lecture 7, CS61B, Fall 2025

Resizing ArrayLists

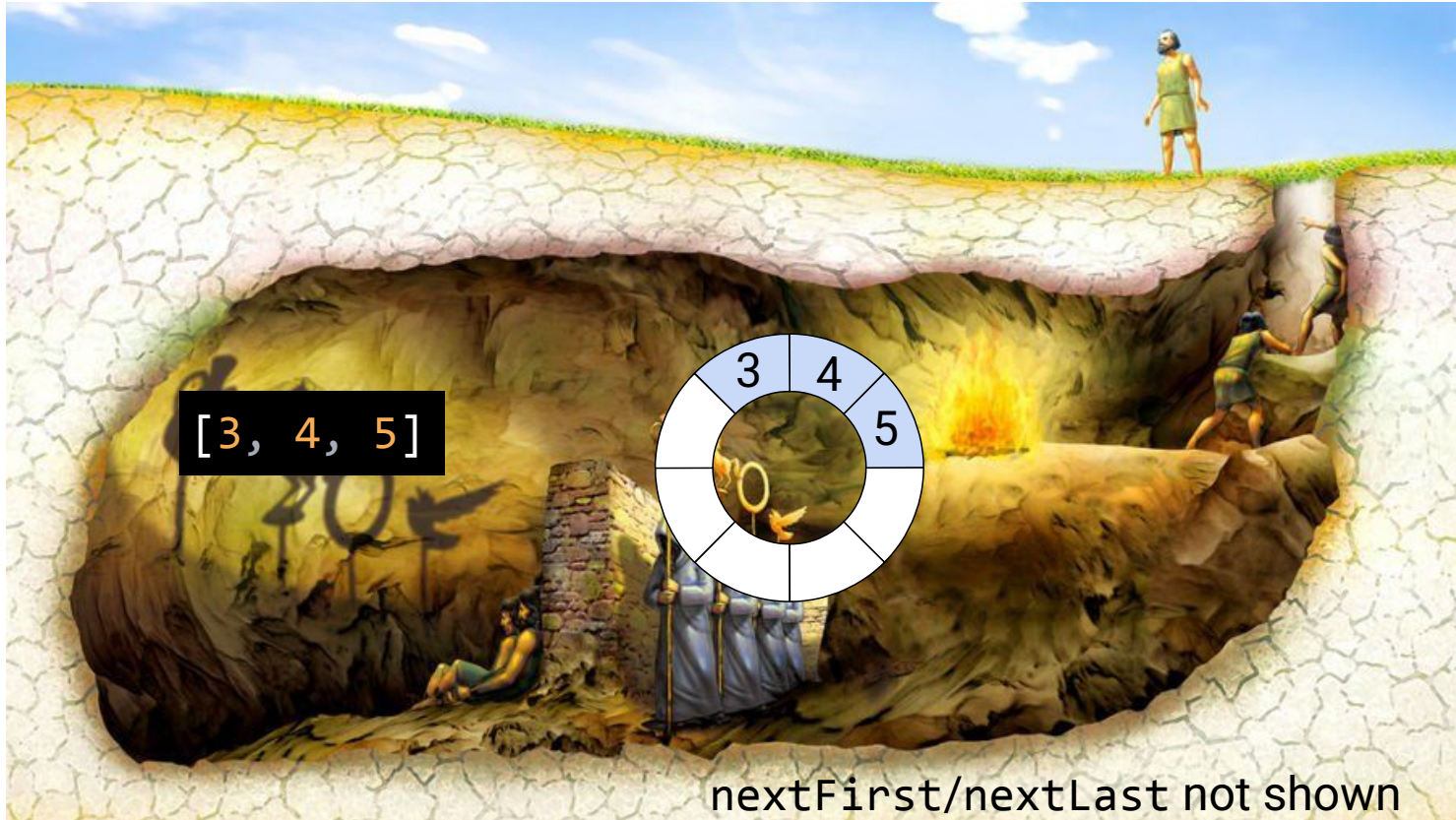
- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Circular ArrayLists

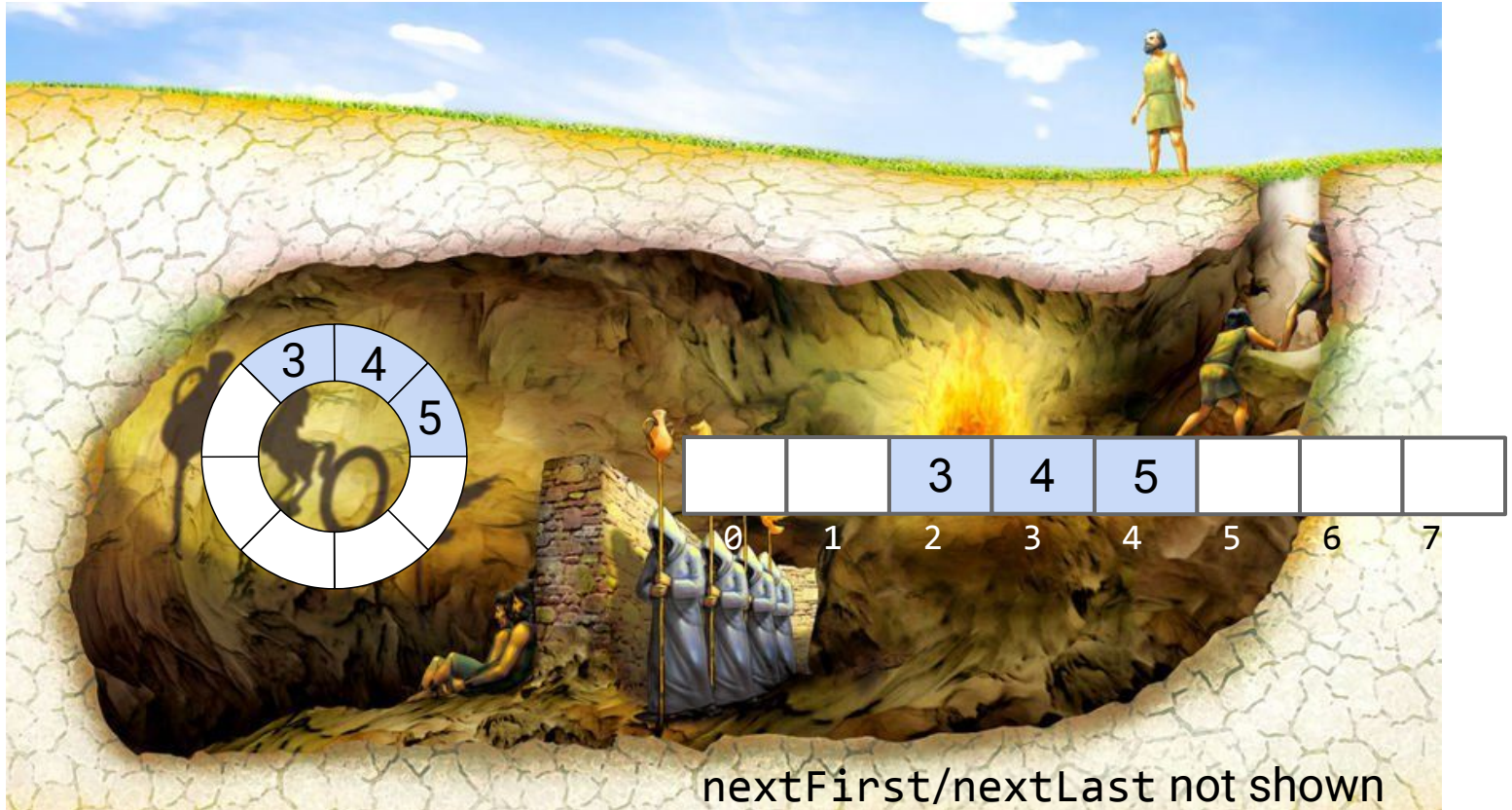
- Circular Arrays
- **Linear Implementation of Circular Arrays**

Circular arrays would be great for implementing abstract lists, but don't exist.

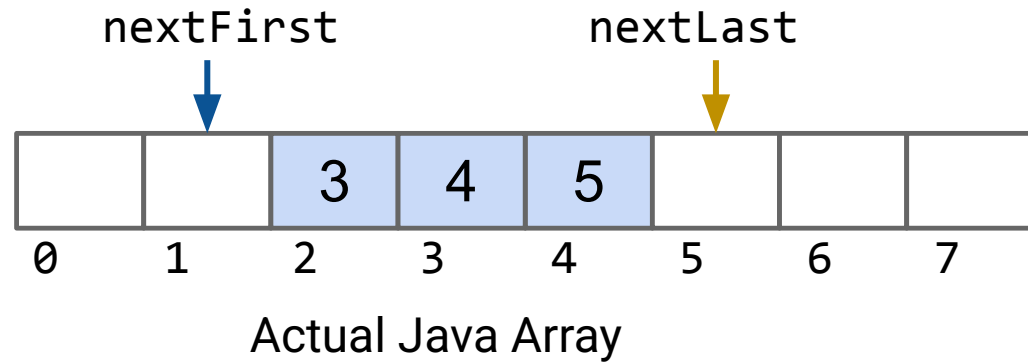
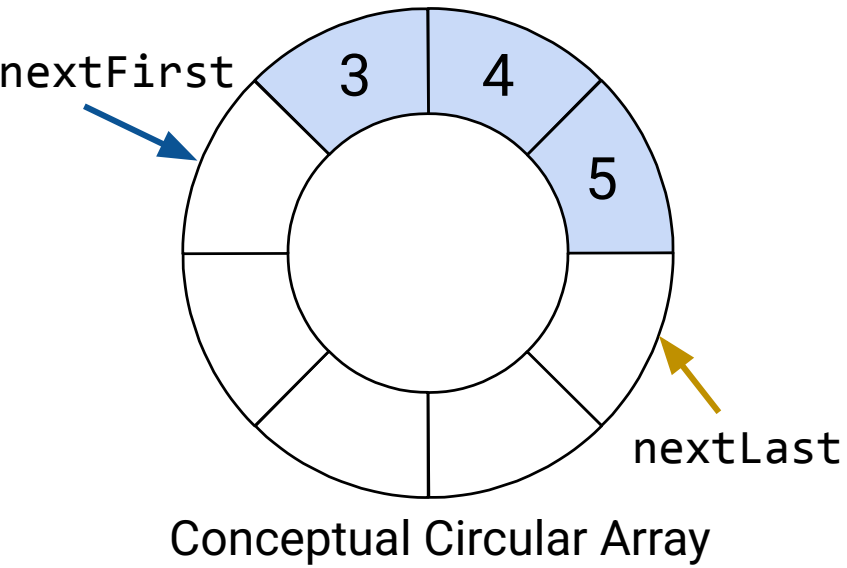


Layers of Abstraction

So we can create the circular array abstraction out of a normal linear array.

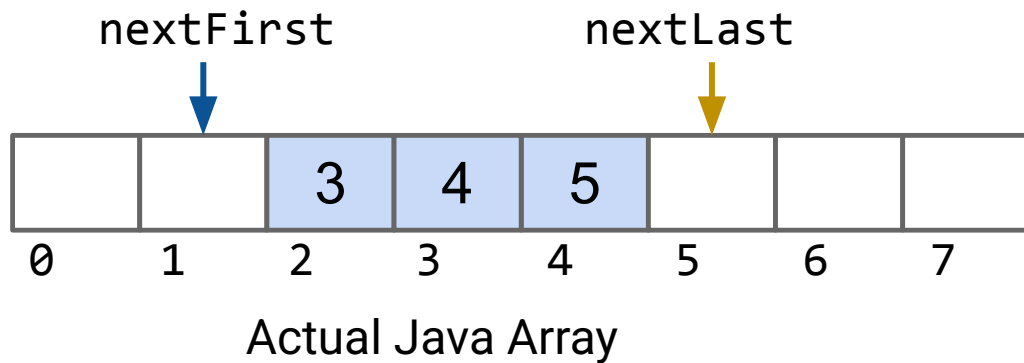
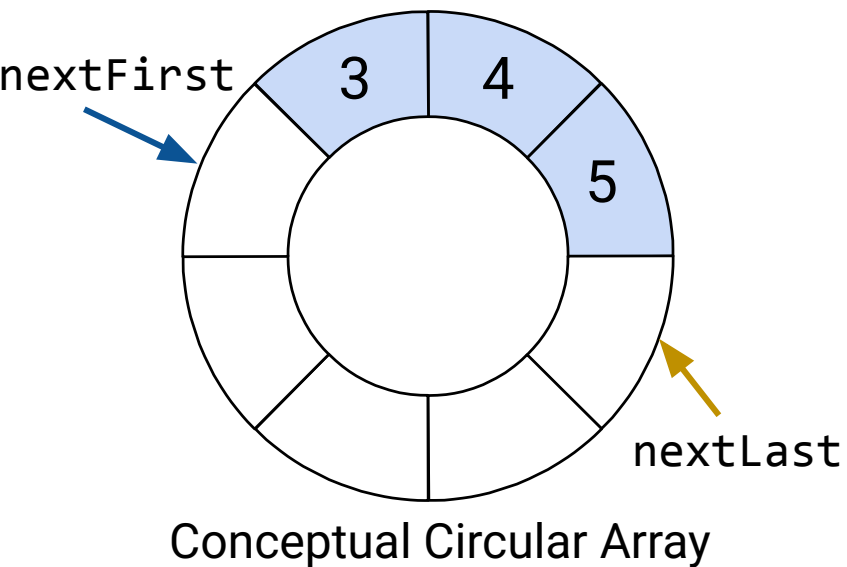


Implementation of a Circular Array



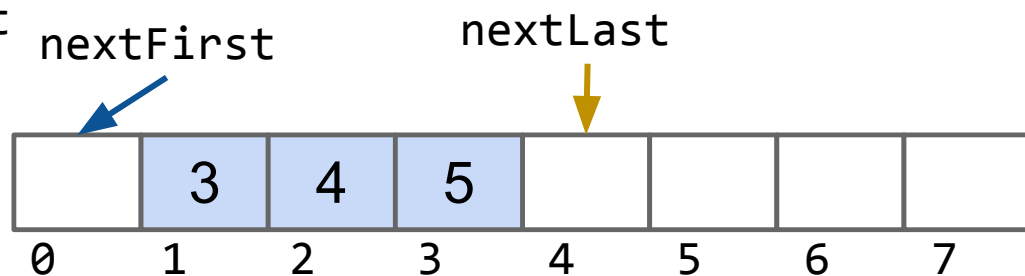
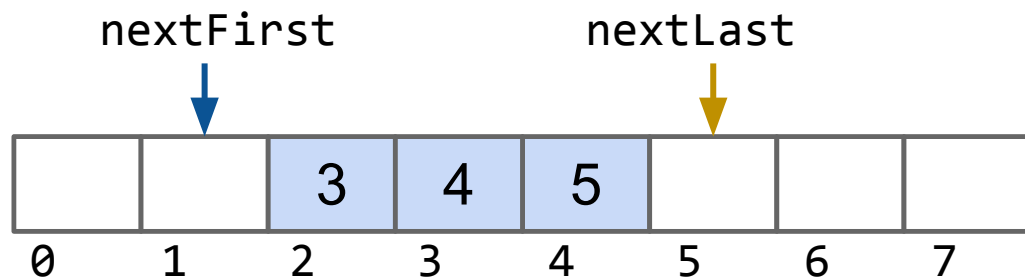
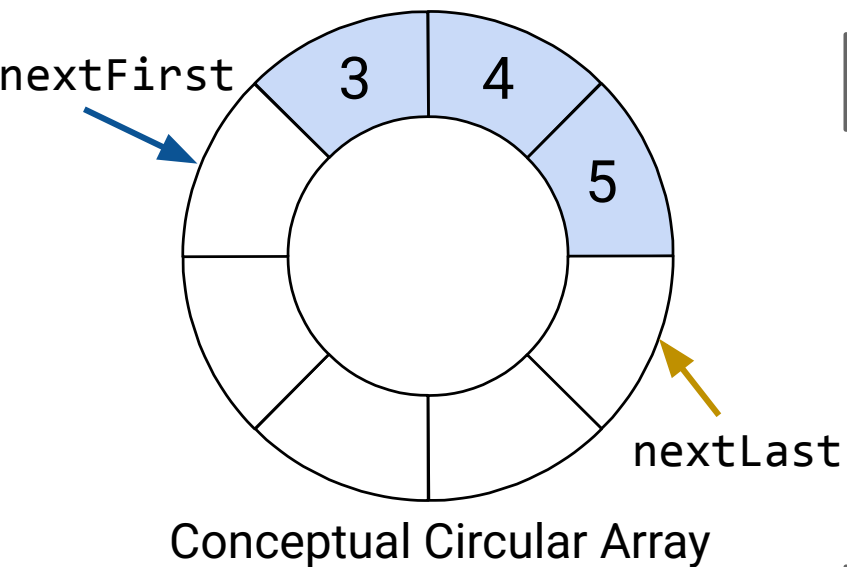
Test Your Understanding

Is the representation unique? That is, are there other actual Java arrays that could represent the conceptual circular array?



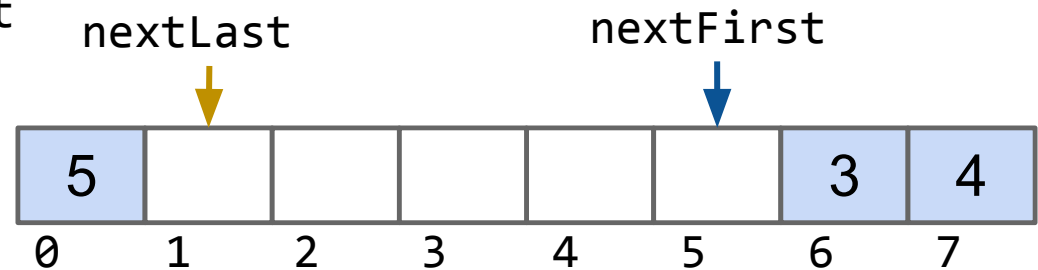
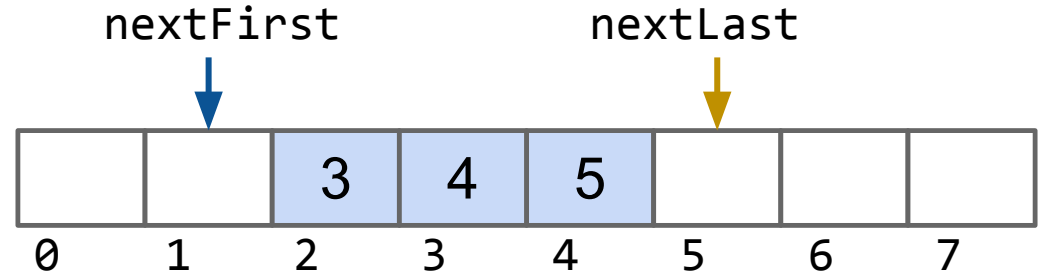
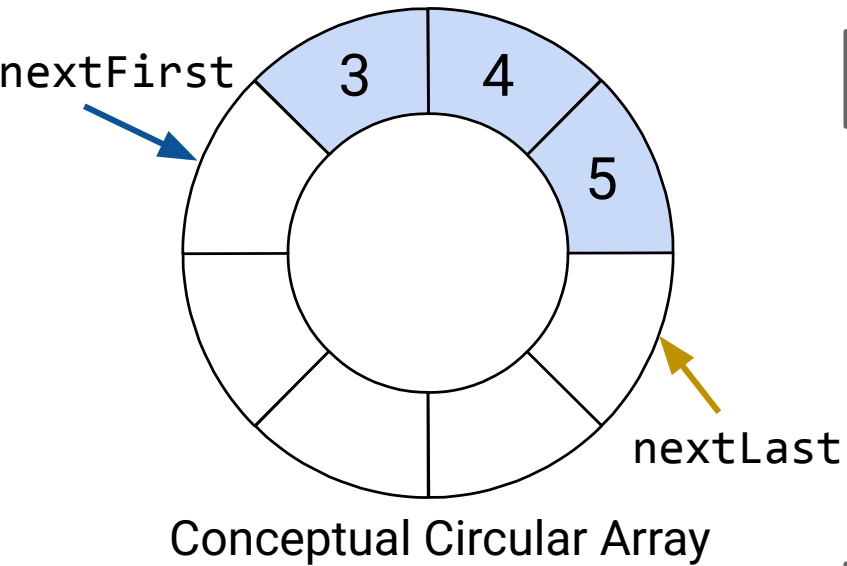
Test Your Understanding

Is the representation unique? No!



Test Your Understanding

Is the representation unique? No!

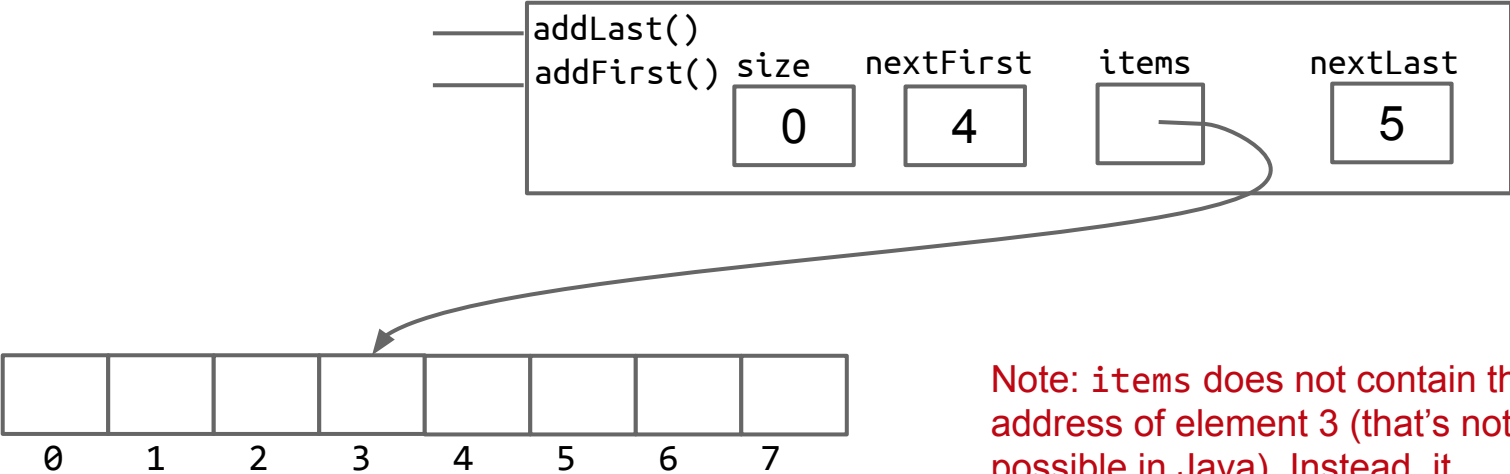


Let's work out an example showcasing how the linear array can be used to represent an abstract list.

- You'll want to look back at these slides when you work on Project 2!

Starting from an empty ArrayList:

Conceptual List: []



Note: `items` does not contain the address of element 3 (that's not possible in Java). Instead, it contains the address of the entire array!

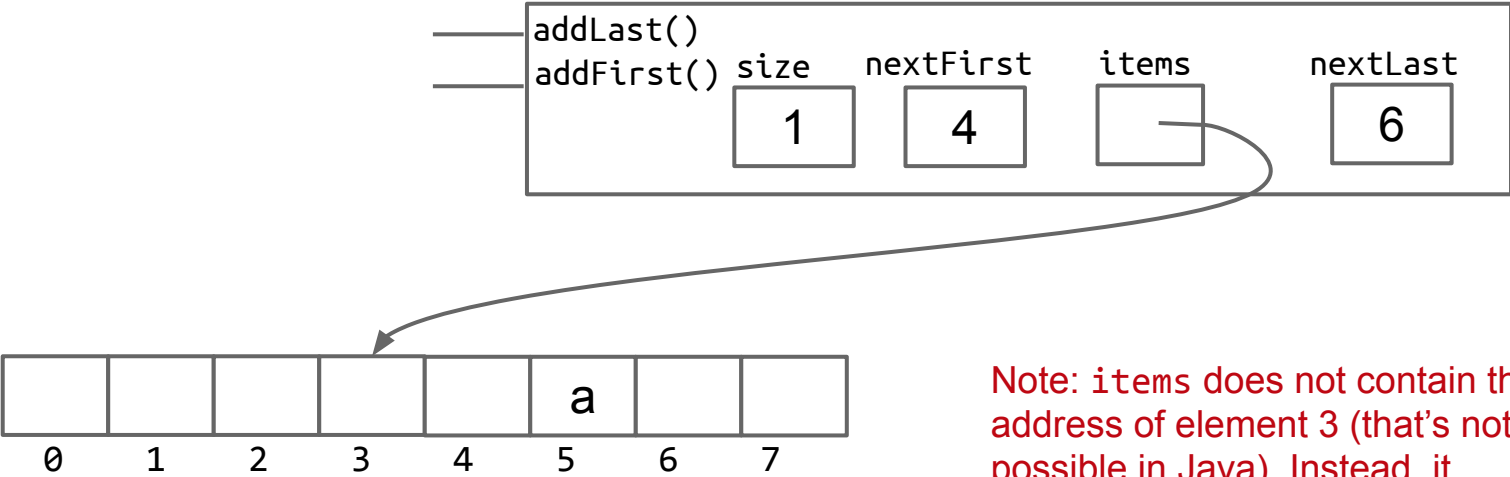
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`

Conceptual List: [a]



Note: `items` does not contain the address of element 3 (that's not possible in Java). Instead, it contains the address of the entire array!

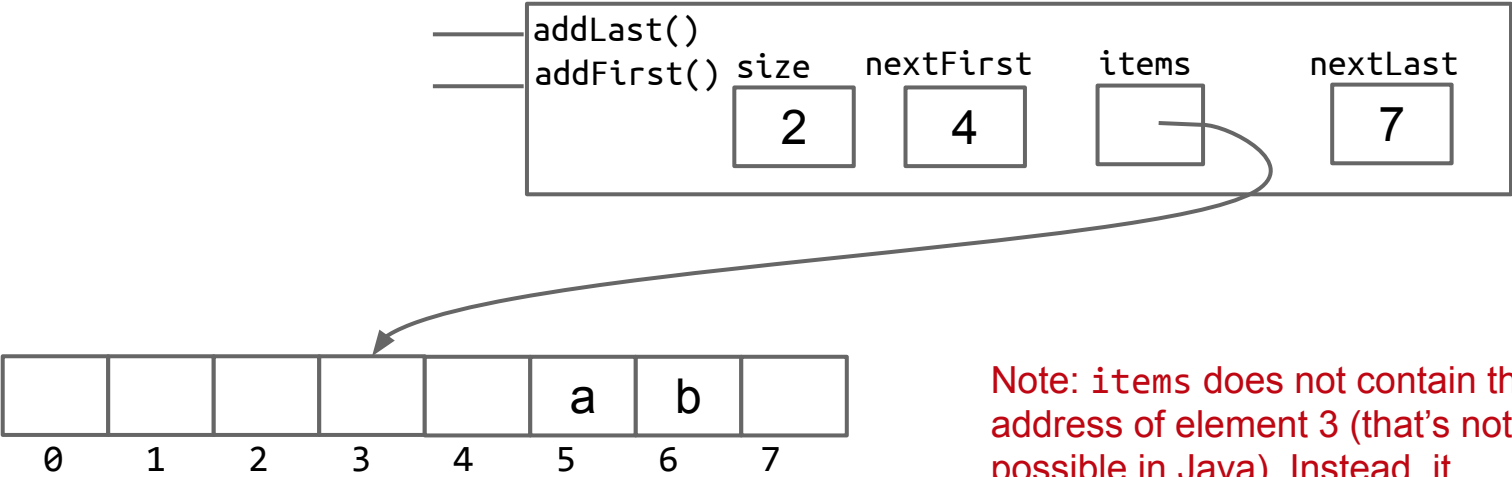
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

Linear Array Demo

Starting from an empty ArrayList:

- addLast("a")
- addLast("b")

Conceptual List: [a, b]



Note: items does not contain the address of element 3 (that's not possible in Java). Instead, it contains the address of the entire array!

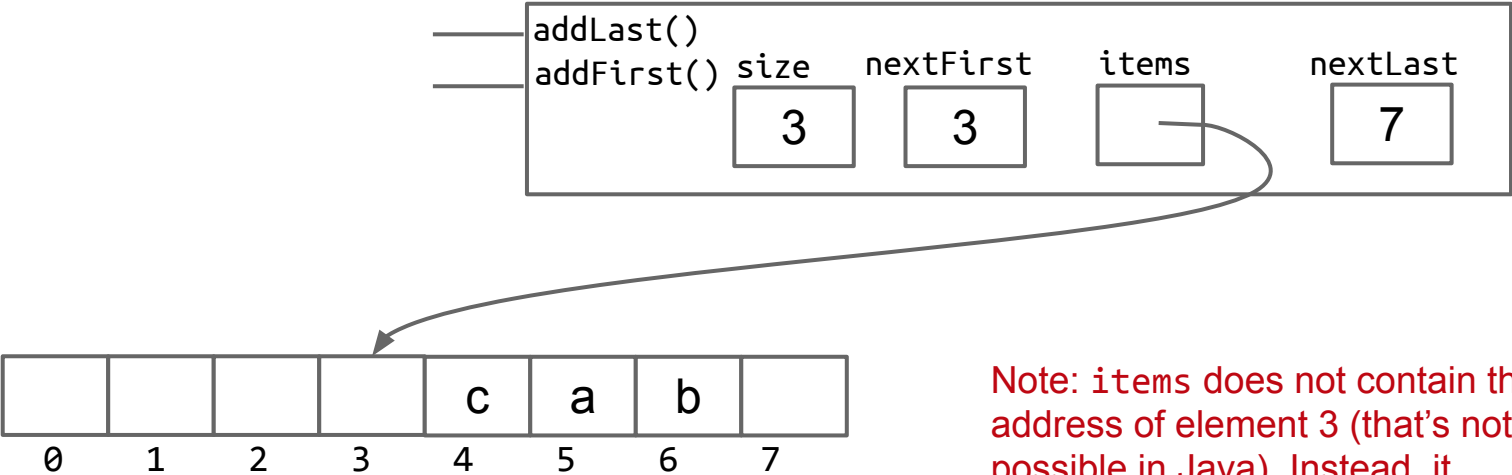
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

Linear Array Demo

Starting from an empty ArrayList:

- addLast("a")
- addLast("b")
- addFirst("c")

Conceptual List: [c, a, b]



Note: `items` does not contain the address of element 3 (that's not possible in Java). Instead, it contains the address of the entire array!

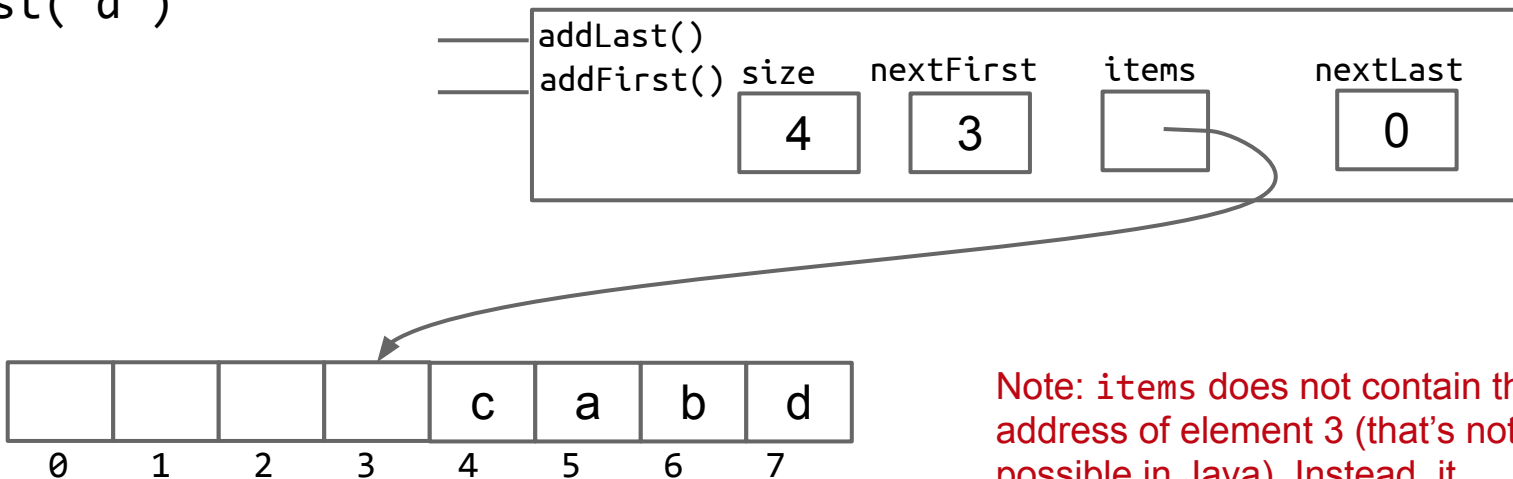
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`

Conceptual List: [c, a, b, d]



Note: `items` does not contain the address of element 3 (that's not possible in Java). Instead, it contains the address of the entire array!

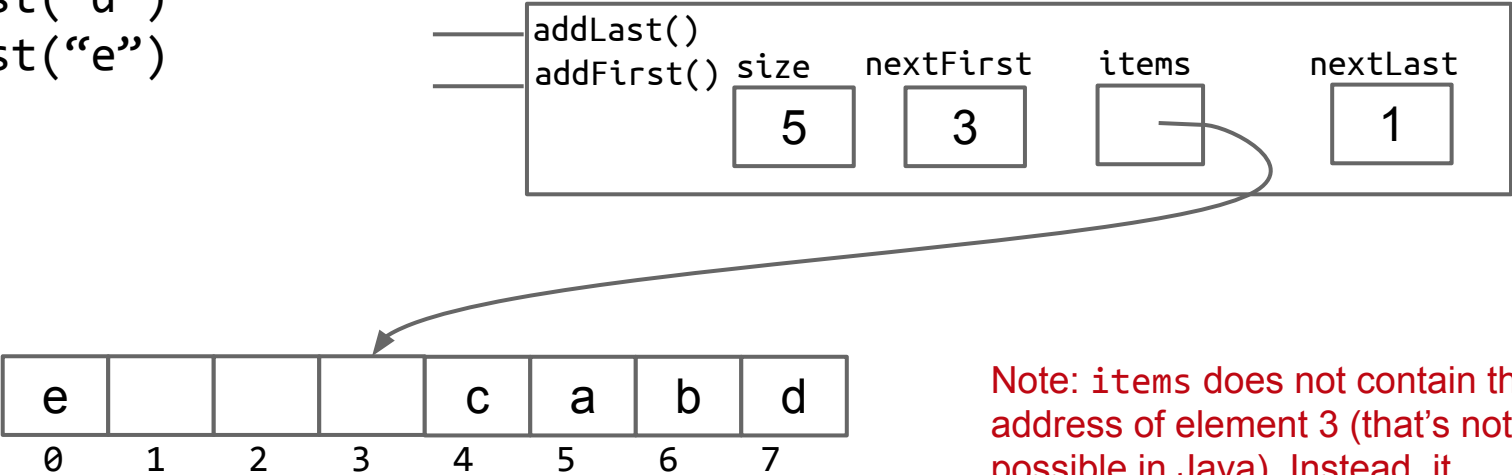
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

Linear Array Demo

Starting from an empty ArrayList:

- addLast("a")
- addLast("b")
- addFirst("c")
- addLast("d")
- addLast("e")

Conceptual List: [c, a, b, d, e]



Note: `items` does not contain the address of element 3 (that's not possible in Java). Instead, it contains the address of the entire array!

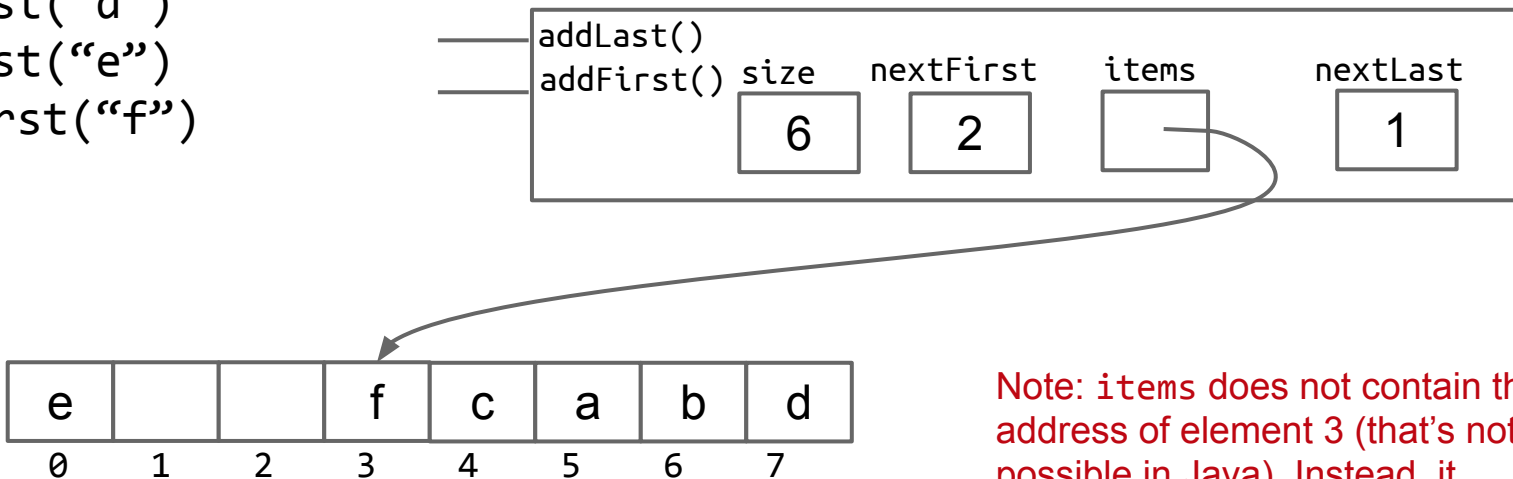
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`
- `addFirst("f")`

Conceptual List: [f, c, a, b, d, e]



Note: `items` does not contain the address of element 3 (that's not possible in Java). Instead, it contains the address of the entire array!

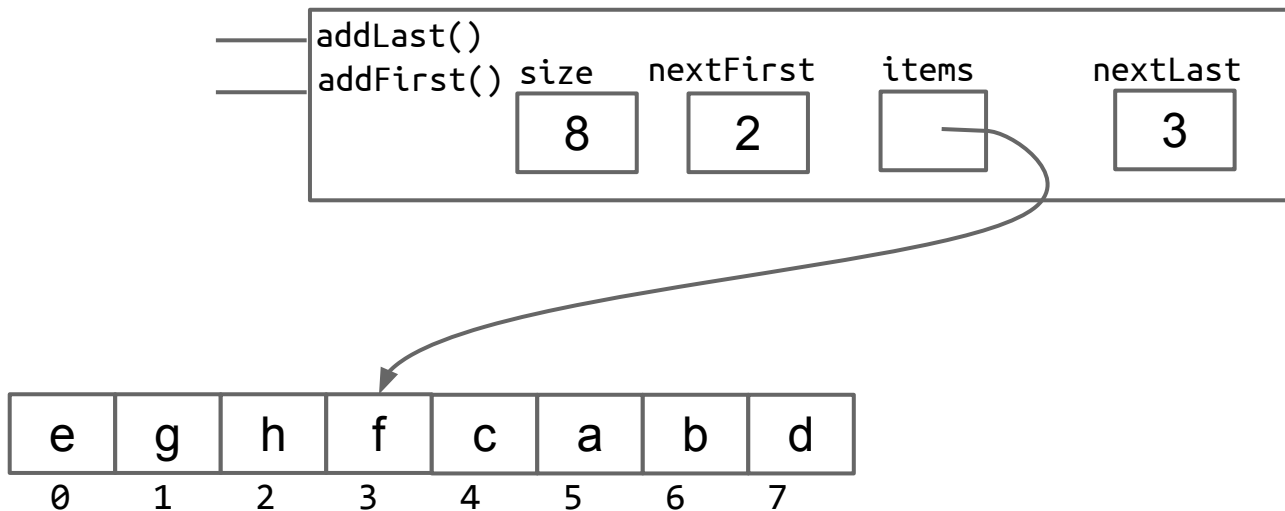
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`
- `addFirst("f")`
- `addLast("g")`
- `addLast("h")`

Conceptual List: [f, c, a, b, d, e, g, h]



After the next **addLast** operation, we'll need a bigger array.

Resizing: You Can Pick Whatever Representation You Prefer for Project 2

Starting from an empty ArrayList:

- addLast("a")
- addLast("b")
- addFirst("c")
- addLast("d")
- addLast("e")
- addFirst("f")
- addLast("g")
- addLast("h")
- **addLast("Z")**

Conceptual List: [f, c, a, b, d, e, g, h, Z]

Before **addLast("Z")**:

e	g	h	f	c	a	b	d
0	1	2	3	4	5	6	7

After **addLast("Z")**: (all of the below are valid choices)

e	g	h	Z								f	c	a	b	d
f	c	a	b	d	e	g	h	Z							
				f	c	a	b	d	e	g	h	Z			

It's your choice what array to use, e.g. all of the three choices above are valid!

- Note: You don't have to use arraycopy if using a for loop is easier for your choice of how to resize.

Take things a little at a time.

- Writing tons of code all at once is going to lead to misery and only misery.
- If you wrote too much stuff and feel overwhelmed, comment out whatever is unnecessary.

Consider not doing resizing at all until you know your code works without it.

- Resizing is a performance optimization (and is required for full credit).
- Recommendation: Code that uses `arraycopy` can be hard to get right.
- Observation: You might find the `get` method useful in your `resize` method.

If your first try goes badly, don't be afraid to scrap your code and start over.

- There isn't that much code.

Python lists do NOT use this circular abstraction.

- `x.insert(0, "hi")` is slow for Python lists.

Why do you think this is?

Python lists do NOT use this circular abstraction.

- `x.insert(0, "hi")` is slow for Python lists.

Why do you think this is?

- Insertion at the front of a list is too rare to justify increased code complexity and performance penalty for other operations.
 - Example: e.g. `x[3:9]` might have `x[3:6]` at the end of the underlying array, and `x[6:9]` at the beginning.

Selecting the Right Data Structure

Users who want fast front operations should use the appropriate data structure.

- `LinkedList` instead of `ArrayList` (in Java).
- `collections.deque` instead of a `list` in Python.
 - Variant of a list with fast front and back operations.

Your project 1 and project 2 solutions are required to have fast front operations (and back operations).

- For your `ArrayDeque`, you are required to use this circular approach.

(Fun (?) fact: Python's built-in `Deque` class is built using a linked list, not a `Deque`. For more see https://stackoverflow.com/questions/45134139/why-is-deque-implemented-as-a-linked-list-instead-of-a-circular-array?utm_source=chatgpt.com)

Attendance link: <https://www.yellkey.com/can>

To-do:

- HW6 due today.
- Project 1 due Monday.
- Get started on Project 2, due September 22.



Linear Array Demo

